

RoboRide Project Report



Engineering Project – 3 Group 2

Bilal Hussain - #9042866

Nikunj Patel - #9047741

Professors:

Safat Khan

Reda Fayek

Submission date: - 10/11/2025

Abstract

The RoboRide project presents the design, construction, and enhancement of a Raspberry Pi-powered robotic vehicle developed across two phases. In Phase 1, the foundation of the system was established through the integration of the Raspberry Pi, L293D motor driver, and dual DC motors to achieve forward, backward, and rotational motion with adjustable speed control. A C-based control program using WiringPi, SoftPWM, and ncurses enabled precise PWM regulation and responsive keyboard-based interaction. Iterative testing resolved issues related to wiring, polarity, and motor consistency, resulting in a stable and reliable motion platform.

Phase 2 expanded RoboRide from a basic motion system into an intelligent, sensor-aware robot. The HC-SR04 ultrasonic sensor was integrated for real-time distance measurement, enabling automated collision prevention during reverse movement. A Raspberry Pi Camera Module 2 was also added, allowing the robot to capture photos and videos while ensuring safe operation through automatic motor pausing. Sensor timing, echo-processing, and error-handling mechanisms were implemented to maintain stable readings under practical conditions.

Together, these phases transformed RoboRide into a versatile, interactive robotics platform that demonstrates core principles of embedded systems, motor control, real-time sensing, and hardware–software integration. The final system illustrates strong potential for use in autonomous navigation, monitoring, education, and future robotics innovation.

Table of Contents

3. Introduction	5
3.1 Project Background	5
3.2 Phase 1 Objectives:	5
3.3 Phase 2 Objectives:	6
3.4 Scope:	6
3.5 Report Overview	7
4. Project Requirements	9
4.1 Functional Requirements	9
4.2 Non-Functional Requirements	10
4.3 Technical Requirements	11
5. Materials, Components and Setup	12
5.1 Hardware Components	12
5.2 Hardware Environment Setup	14
5.3 Software Tools	18
5.4 Software Environment Setup	19
5.5 Software Setup Procedure	20
5.6 Cost Analysis	21
6. Software Requirements Specification (SRS)	22
7. Software Design Description (SDD)	26
8. Circuit Design	33
8.1 Circuit Diagram	33
8.2 Schematic Diagram	34
8.4 Wiring Description	35
9. Implementation	36
9.1 Code Structure	36
9.2 Key Code Snippets	37
9.4 Challenges and Solutions	42
10. Testing and Results	43
10.1 Testing Plan	43
10.2 Test Results	49
10.3 Observations	50
11. Discussion	51

RoboRide Report

11.1 Analysis of Results	51
11.2 Limitations.....	51
11.3 Potential Improvements.....	51
12. Conclusion	52
13. References.....	53
14. Appendices.....	54
14.1 Full Code Listing.....	54
14.2 Explanation of code for better understanding	62
14.3 Additional Diagrams or Tables	64

3. Introduction

3.1 Project Background

The RoboRide project was developed to explore fundamental concepts in embedded systems, robotics, and motion control through a practical, low-cost platform. In Phase 1, a Raspberry Pi was integrated with an L293D motor driver and dual DC motors to create a robotic chassis capable of forward, reverse, and turning motions. PWM-based speed control was implemented to provide smooth and reliable actuation, emphasizing hardware–software integration and the importance of precise motor control.

Phase 2 added an HC-SR04 ultrasonic sensor for obstacle detection and a Raspberry Pi Camera Module for image and video capture, giving the robot basic autonomy and perception to sense its environment.

Together, these phases transformed RoboRide from a simple motorized platform into an interactive, sensor-driven system that demonstrates key principles of mobile robotics, including movement, sensing, safety, and real-time response.

The aim of the RoboRide project is to design and implement a cost-effective robotic platform using Raspberry Pi that integrates motor actuation, ultrasonic sensing, and camera-based perception. The project seeks to achieve safe navigation, responsive control, and interactive functionality, thereby providing a hands-on demonstration of embedded systems and robotics fundamentals.

3.2 Phase 1 Objectives:

- Develop a functional Raspberry Pi–based robotic car capable of controlled movement.
- Implement direction and speed control using an L293D motor driver and PWM signals.
- Integrate hardware components including motors, batteries, and wiring on a chassis platform.
- Test and validate performance through motion experiments and oscilloscope measurements.
- Analyze challenges related to wiring congestion, motor polarity, and power supply limitations.

3.3 Phase 2 Objectives:

- Integrate the HC-SR04 ultrasonic sensor to enable real-time distance measurement and obstacle awareness.
- Add the Raspberry Pi Camera Module 2 for image and video capture on user command.
- Modify the control software to support continuous sensor reading, automatic reverse safety-stop, and camera operations.
- Improve system reliability by implementing timeout handling, noise filtering, and periodic distance sampling.
- Test and validate sensor performance through controlled measurements, temperature variation testing, and distance accuracy comparison.
- Document hardware additions including mounting, wiring adjustments, and new power/signal routing for the sensor and camera.
- Enhance the user interface to include clear status messages, obstacle alerts, and camera-operation feedback.

3.4 Scope:

The scope of this project includes the design, wiring, and software implementation of a robotic vehicle that can move forward, backward, left, and right under manual command. The system utilizes a Raspberry Pi for control logic and an L293D driver for motor interface. The project focuses on establishing reliable wired control, power management, and PWM testing.

However, it does not cover autonomous navigation, wireless communication, These features are identified as possible future expansions that would increase the robot's intelligence and adaptability.

3.5 Report Overview

This report is organized to present the full development cycle of the RoboRide robotic system, covering both the foundational Phase 1 build and the advanced Phase 2.

- **Introduction**
Provides an overview of the RoboRide project, its motivation, objectives, and relevance. It outlines the transition from a basic motor-controlled robot in Phase 1 to an intelligent sensing and camera-enabled system in Phase 2.
- **Project Requirements**
Defines the functional, non-functional, and technical requirements across both phases, including movement control, speed regulation, sensor integration, safety features, and camera functionality.
- **Materials and Components**
Details all hardware components used throughout the project: Raspberry Pi Zero W, L293D motor driver, motors, batteries, HC-SR04 ultrasonic sensor, Pi Camera Module 2, and additional wiring/mounting supplies. Includes updated cost analysis and all software tools (WiringPi, ncurses, Raspberry Pi OS, VNC, libcamera, etc.).
- **Software Requirements Specification (SRS)**
Describes the software functionalities such as PWM-based motor control, ncurses terminal interface, ultrasonic distance measurement, obstacle-aware safety stop, and camera operations (photo/video capture). Also includes system constraints, assumptions, and performance expectations.
- **Software Design Description (SDD)**
Explains the complete software architecture from Phase 1 to Phase 2, including modular design for motor control, sensor measurement timing loops, distance filtering, camera command handling, and ncurses-based UI design. Discusses global variables, function interactions, and data handling.
- **Circuit Design**
Presents wiring diagrams for motors, L293D driver, power distribution, and Phase 2 additions such as the HC-SR04 with voltage divider and Raspberry Pi Camera Module. Includes pin tables, justification for wiring decisions, and safety notes related to power sharing and signal routing.
- **Implementation**
Covers how the system was built in practice:
 - Motor control and PWM tuning (Phase 1)
 - Sensor integration using busy-wait loops and timeout logic (Phase 2)
 - Real-time UI implementation using ncurses
 - Camera image/video capture functionality via libcamera-still and libcamera-vid

- Integration challenges and solutions
Provides annotated code snippets and explains how modules work together.
- **Testing and Results**
Includes testing from both phases:
 - Motor tests, PWM oscilloscope measurements, motion accuracy (Phase 1)
 - Distance measurement accuracy, 250 ms sampling tests, timeout behavior, temperature-dependent variations (Phase 2)
 - Camera functionality tests (photo, video, UI blocking behavior, stabilization)Compares expected vs. actual results with observations.
- **Discussion**
Analyzes the full system performance:
 - Reliability of motor control and sensor readings
 - Handling of noise, drift, and environmental factors
 - Impact of wiring congestion, motor torque imbalance, and battery performance
 - Strengths, limitations, and integration challenges from both phasesReflects on technical improvements made and lessons learned.
- **Conclusion & Future Work**
Summarizes achievements across both phases, highlighting the evolution from basic movement control to an advanced sensing and camera-enabled robot.
Suggests meaningful future improvements such as wireless remote control, AI-based object detection, enhanced autonomy, and chassis upgrades.
- **References & Appendices**
Includes all citations, datasheets, wiring diagrams, test results, oscilloscope captures, code listings, and logs (e.g., distance_log.txt). Also contains camera images, screenshots, and additional documentation supporting the project.

4. Project Requirements

4.1 Functional Requirements

The *RoboRide* system is designed to perform essential motion and control functions through the integration of hardware and software components. The primary functional requirements are as follows:

- **Motor Control**
The system must control two DC motors through the L293D driver to enable forward, backward, left-spin, and right-spin movement based on user commands.
- **PWM Speed Control**
The Raspberry Pi must generate PWM signals to switch between slow and fast motor speeds for smoother and more efficient movement.
- **Ncurses User Input**
The robot must respond immediately to non-blocking keyboard commands (W, A, S, D, 1, 2, Space, Q) through a ncurses-based terminal interface.
- **Ultrasonic Distance Measurement (HC-SR04)**
The robot must measure distance every 250 ms using a timed busy-wait loop and return stable values within the 2–400 cm range. Invalid or timeout readings must be detected and handled.
- **Reverse Safety Stop**
When moving backward, the robot must automatically stop if an obstacle is detected within a set threshold (e.g., <20 cm). Forward motion remains fully manual.
- **Camera Operation**
The system must support capturing photos and videos via terminal commands. The robot must stop before taking a photo or recording video for safety.
- **Motor Safety Handling**
Before changing direction or speed, the system must briefly stop both motors to prevent electrical stress or sudden mechanical strain.
- **Dual Power Management**
The robot must operate using two power sources — a power bank for the Raspberry Pi and AA batteries for the motors — with a shared ground connection.
- **Expandability**
The software must be structured so future enhancements (additional sensors, wireless control, autonomous features, etc.) can be integrated without major redesign.

4.2 Non-Functional Requirements

The *RoboRide* system must not only perform its intended functions but also meet key standards of performance, reliability, and usability. The following non-functional requirements ensure that the robotic vehicle operates efficiently, safely, and consistently under typical use conditions:

- **Performance Requirements**
 - The robot must respond to terminal commands within 1 second.
 - PWM signals must remain stable to ensure smooth speed transitions and consistent motion.
 - Ultrasonic distance readings must update every 250 ms without affecting robot movement or camera functions.
- **Reliability Requirements**
 - The system must operate continuously for at least 30 minutes without overheating, unstable voltage, or motor drop-outs.
 - Repeated start-stop motor cycles must not affect functionality or cause L293D driver faults.
 - Dual-power setup (Pi + AA motor pack) must maintain stable voltage with a common ground shared across the system.
- **Usability Requirements**
 - The ncurses interface must allow simple real-time control using single-key commands (W, A, S, D, 1, 2, Space, Q).
 - All actions—movement, ultrasonic readings, camera capture, and auto-stop—must clearly display feedback on screen.
 - Component layout should remain accessible for easy wiring checks, hardware replacement, and new sensor integration.
- **Efficiency Requirements**
 - PWM control must reduce unnecessary motor load to extend AA battery life.
 - Sensor reading, camera commands, and motor control must run efficiently without high CPU usage or thermal throttling on the Raspberry Pi Zero W.
 - Ultrasonic timing loops must use lightweight busy-wait logic to avoid blocking other system functions.

4.3 Technical Requirements

Category	Requirement Description
Platform Requirements	Must operate on a Raspberry Pi Zero W running Raspberry Pi OS, supporting motor control, ultrasonic sensing, and camera functions simultaneously without system instability.
Interface Requirements	Must interface with the L293D motor driver, HC-SR04 ultrasonic sensor, and Raspberry Pi Camera Module 2 using appropriate GPIO pins and communication protocols.
Power System Requirements	Must maintain stable dual-power operation: Pi powered by power bank and motors powered by AA battery pack, while also supporting additional sensor + camera load without voltage drop.
Control Input Requirements	Movement commands (W, A, S, D, STOP, Speed 1/2) and camera commands (photo, video) must be entered through an ncurses-based terminal UI, reacting in real time.
Output Requirements	The robot must perform corresponding actions (movement, stopping, auto-halt when reversing, distance measurement, camera capture) with clear on-screen UI feedback.
Speed Control Requirements	Must implement PWM-based variable speed control with smooth transitions between speed presets, even while sensor readings and UI updates run in parallel.
Wiring & Circuit Requirements	Must maintain correct GPIO mapping for motors, sensor, and camera with proper level shifting for the ECHO pin and secure cable routing to prevent movement interference.
Safety Requirements	Must automatically stop the robot when reversing if an object is detected within a set threshold distance (e.g., < 15 cm), ensuring collision prevention.
Documentation Requirements	Must include setup instructions for sensor integration, camera configuration, updated wiring diagrams, timing logic, and logging features (distance logging).
Future Expansion	System should support extensions including full autonomous navigation, machine-vision obstacle detection, wireless control, and cloud-based camera streaming.

Table 1: Technical requirements for RoboRide Project

5. Materials, Components and Setup

5.1 Hardware Components

The *RoboRide* prototype integrates several electronic and mechanical components to achieve motor control, power management, and system stability. The table below lists all hardware components used in the project.

No.	Components	Pictures	Qty	Description
1	Raspberry pi 0 W		1	Acts as the brain of the system, processing input commands and generating PWM signals to control motor speed and direction.
2	L293D Motor Driver IC		1	Serves as the interface between the Raspberry Pi and the DC motors, providing the necessary current and voltage to drive the motors.
3	DC Motors		2	Provide motion to the robotic vehicle; connected to the driver for bidirectional control.
4	Car Chassis		1	Mechanical base that holds the motors, driver, and Raspberry Pi; enables movement on flat surfaces.
5	Breadboard		1	Used for prototyping and connecting electronic components without soldering.
6	Jumper Wires		~20	Facilitate electrical connections between the Raspberry Pi, motor driver, and other components.
7	Power Bank (5000mah)		1	Provides a stable 5v power supply to the Raspberry Pi.
8	4 * AA Battery pack		1	Powers the DC motors through the motor driver IC, ensuring isolated motor operation.

RoboRide Report

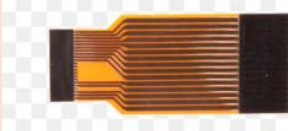
9	Wheels/Tyres		2	Convert motor rotation into linear motion for vehicle movement.
10	USB – Micro USB cable		1	Used to connect and power the Raspberry Pi from the power bank.
11	HC-SR04 (Ultrasonic Sensor)		1	8 MP camera for capturing images and video; connects via CSI port.
12	Raspberry pi Camera Module 2		1	Ultrasonic distance sensor (2–400 cm range) for obstacle detection.
13	PI 0 W Camera Cable		1	Pi 0 W needs 22 pin cable, this cable helped to connect camera to the pi.

Table 2: Hardware Components List and Brief Description

5.2 Hardware Environment Setup

1. Assemble the 2WD Car Chassis

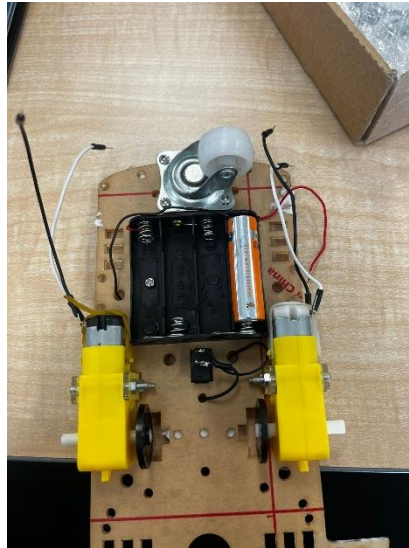


Figure 1: 2WD Chassis setup picture.

As shown in Figure 1, Follow the chassis kit manual to mount the two DC motors, attach the wheels, and fix the acrylic/PCB base plate. Install the AA battery holder on the chassis, positioned to keep the center of gravity balanced.

2. Mount the Breadboard and Raspberry Pi Zero W

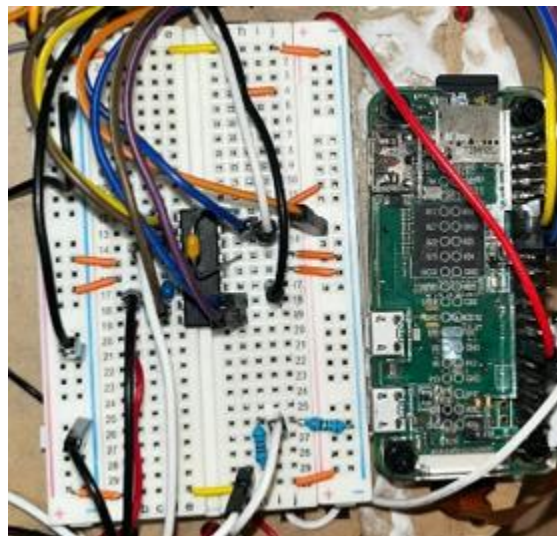


Figure 2: Breadboard and Pi Setup picture.

As shown in Figure 2, Fix the small breadboard on the top of the chassis using double-sided tape. Mount the Raspberry Pi Zero W beside it, ensuring the GPIO header is accessible for wiring and the USB/power ports are reachable.

3. Install the L293D Motor Driver on the Breadboard

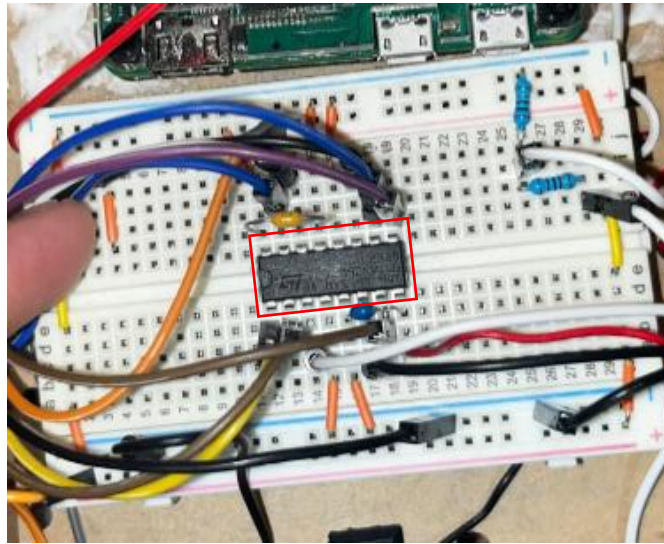


Figure 3: L293D Placement Picture.

As marked by red box in the Figure 3, Place the L293D IC in the middle of the breadboard, straddling the center gap. Leave enough space around it for motor, power, and GPIO connections.

4. Wire Motors to the L293D Outputs

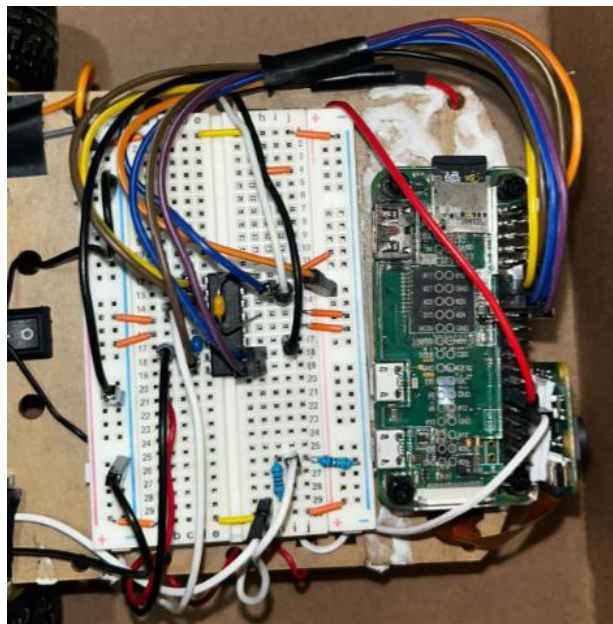


Figure 4: Wiring for L293D IC.

Refer the Wiring Map from Table 6 (Page 35) and Circuit Diagram Figure 9 (Page 33). Connect the motors, ground, Raspberry Pi the IC. Secure the motor wires using tape so they do not get tangled with the wheels during motion. (Figure 4)

5. Install Power Bank on the Chassis

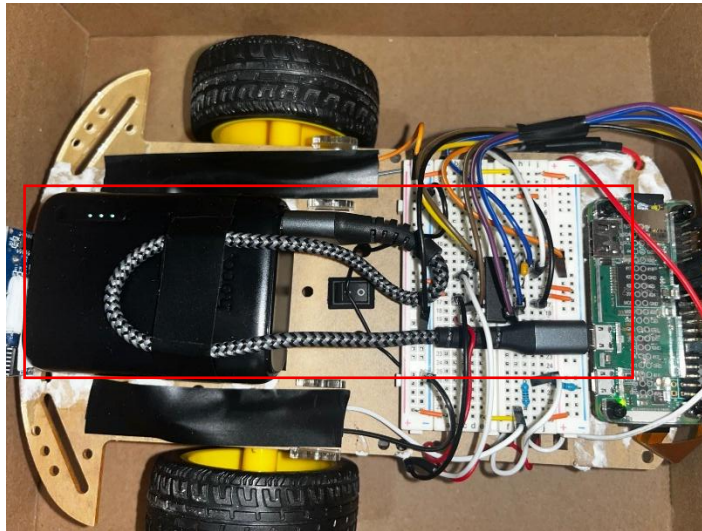


Figure 5: Picture of Power Bank from RoboRide.

As shown in Figure 5, place the Power bank using the double tape on the Chassis, the USB port must be facing the raspberry pi. Connect the raspberry pi to the power bank using a short armor cable. Make sure to have it intact and leave the gap at front for the ultrasonic sensor.

6. Place and Wire the Ultrasonic Sensor (HC-SR04)



Figure 6: Ultrasonic Sensor mounted on the Robot.

As shown in Figure 6. Mount the HC-SR04 at back of the robot, pointing directly backward for reverse obstacle detection. Connect TRIG and ECHO to the assigned GPIO pins via the breadboard, adding the resistor voltage divider on the ECHO line to safely step 5 V down to 3.3 V for the Pi. Refer Table 6 (Page 35) for Wiring help.

7. Install and Connect the Camera Module 2

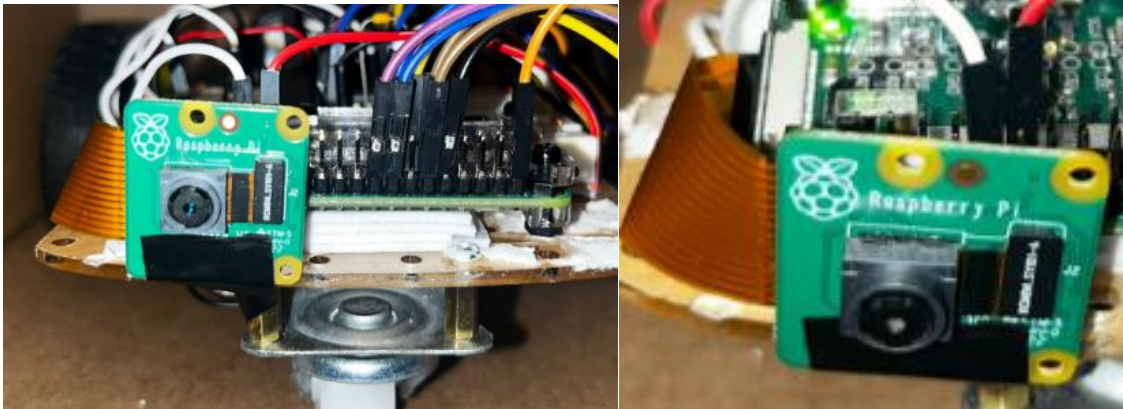


Figure 7: Raspberry Pi Camera mounted on the Robot.

Attach the Raspberry Pi Camera Module 2 at the front of the robot, aligned with the forward direction of travel. Carefully insert the camera ribbon cable into the Pi Zero W CSI connector, making sure the contacts face the correct direction, the copper sides must face the PCB and the latch is fully closed.

8. Cable Management and Final Checks

Secure all jumper wires and the battery leads with tape so they do not interfere with the wheels or sensors. Check Polarity of the motors. Finally, verify every connection against the wiring table before powering the system for software testing.

5.3 Software Tools

The *RoboRide* project made use of several software tools and libraries for programming, simulation, circuit design, and testing. These tools were selected for their accessibility, reliability, and compatibility with the Raspberry Pi environment. The table below summarizes each tool, its purpose, license type, and cost.

No.	Software Tool	Description	License Type	Cost/Link	Icon
1	Terminal Interface	Used to write, compile, and execute C programs directly on the Raspberry Pi. Also provides debugging features for testing motor commands and PWM control.	Open Source (MIT License)	Free	
2	WiringPi Library	A GPIO control library that enables communication between the Raspberry Pi and external components such as the L293D motor driver. [22]	Open Source (GNU GPLv3)	Free	WiringPi
3	SoftPwm Library	Provides software-based Pulse Width Modulation (PWM) functionality for the Raspberry Pi, used for controlling motor speed.	Open Source (GNU GPLv3)	Free	SoftPwm
4	Raspberry pi OS	The official operating system for Raspberry Pi, used to execute code, manage GPIO, and handle power and process control.	Open Source (GNU GPLv2)	Free	 Raspberry Pi
5	Fritzing	Used to design and simulate the circuit diagram before physical implementation, ensuring accurate wiring of the L293D motor driver and Raspberry Pi pins.[10]	Open Source (GPL License)	Free	
6	Oscilloscope	Used to observe PWM waveforms and confirm correct signal duty cycles for verifying motor speed and performance.	Lab Equipment	Institution	
7	VNC Viewer	This software help to get remote desktop access to the Raspberry Pi.[23]	Free Software Open for all.	Free	

Table 3: Software Tools used for RoboRide Project.

5.4 Software Environment Setup

The RoboRide project was developed using a Linux-based software environment that supported both motor-control programming and sensor-camera integration. Ubuntu Linux served as the primary development platform due to its stability, open-source flexibility, and strong support for embedded systems tools. All code was written in C and compiled using the GNU GCC compiler to ensure portability and low-level hardware control.

For Phase 1, the software stack included the WiringPi and softPwm libraries, which enabled PWM-based motor control and GPIO interfacing with the L293D motor driver. Raspberry Pi OS Lite was installed on the Raspberry Pi Zero W to allow lightweight, headless operation. Communication between the development laptop and the robot was achieved through SSH, enabling fast code transfer, real-time debugging, and command-line testing.

In Phase 2, additional libraries and tools were incorporated to support the HC-SR04 ultrasonic sensor, Raspberry Pi Camera Module 2, and improved terminal UI. The ncurses library was used to create an interactive keyboard-controlled interface, allowing simultaneous robot control and live distance monitoring. The libcamera suite was installed on the Pi to capture images and videos from the camera module. VNC Viewer was also configured to provide a graphical desktop interface for tasks requiring visual output, such as viewing captured images or testing camera functionality. Together, these tools created a complete software ecosystem capable of handling sensing, imaging, motion control, logging, and safety automation in a unified environment.

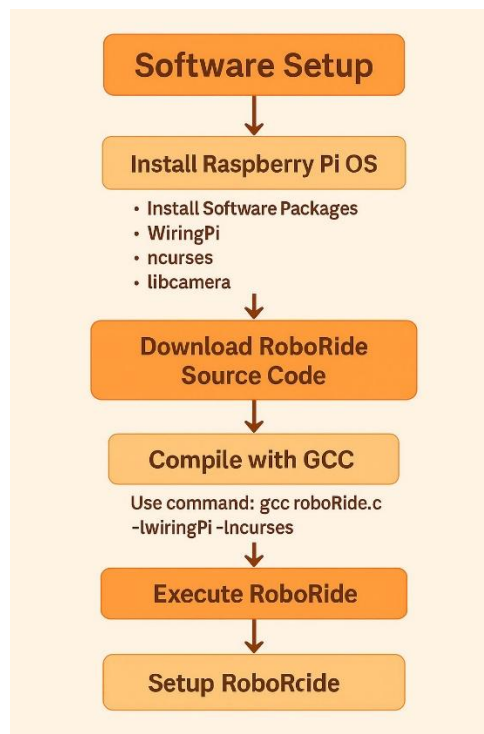


Figure 8: Software Setup Diagram

RoboRide Report

5.5 Software Setup Procedure

The RoboRide setup procedure steps explained using a table:

Step	Task	Description	Tools / Software Used
1	Ubuntu Installation	Installed Ubuntu 22.04 LTS using a bootable USB. Updated packages and prepared development environment.	Rufus, Ubuntu ISO, apt update, apt upgrade, build-essential
2	Raspberry Pi OS Flashing	Flashed Raspberry Pi OS Lite onto microSD. Enabled SSH & Wi-Fi directly in Raspberry Pi Imager.	Raspberry Pi Imager, microSD card
3	Raspberry Pi Power-Up & Connection	Inserted SD card into Pi Zero W, powered via 5V. Connected via SSH through hotspot.	Power bank, SSH terminal, mobile hotspot
4	Network Verification	Ensured Pi was reachable through hostname/local IP.	ping raspberrypi.local, ssh pi@<ip>
5	Core Library Installation	Installed GPIO, PWM, and terminal UI libraries: WiringPi, softPwm, ncurses.	sudo apt install wiringpi, sudo apt install libncurses5-dev
6	Camera Software Setup (Phase 2)	Installed libcamera suite for photo/video capture. Verified camera detection.	sudo apt install libcamera-apps, libcamera-still --list-cameras
7	Ultrasonic Sensor Setup (Phase 2)	Wired TRIG (GPIO5), ECHO (GPIO12 with divider), verified with simple test code.	WiringPi, multimeter, HC-SR04
8	Development Workflow	Wrote and edited code using nano. Compiled using GCC.	nano, gcc -lwiringPi -lncurses ...
9	Integration Testing	Started with motor tests, then added ultrasonic reading tests (2s/250ms), and finally camera actions.	SSH terminal, oscilloscope (PWM validation)
10	Debugging & Verification	Fixed wiring congestion, reversed polarity, ensured clean echo signals, validated camera output.	Serial console logs, distance logs, captured images
11	Final Assembly	Mounted sensor and camera, stabilized wiring with tape, adjusted power layout for stability.	Fritzing diagrams, chassis, jumper wires
12	Full System Final Test	Tested movement, speed control, obstacle auto-stop in reverse, and picture/video capture.	RoboRide C program, Pi camera, HC-SR04

Table 4: Setup Procedure for RoboRide.

5.6 Cost Analysis

No.	Component	Estimated Cost (CAD)
1	Raspberry Pi Zero W	\$60.00
2	L293D Motor Driver IC	\$2.00
3	DC Motors (x2)	\$10.00
4	2WD Car Chassis	\$12.00
5	Breadboard	\$6.00
6	Jumper Wires Set	\$5.00
7	Power Bank (5000 mAh)	\$20.00
8	AA Battery Holder	\$3.00
9	AA Batteries (x4)	\$4.00
10	USB Cable (Micro-USB)	\$4.00
11	HC-SR04 (Ultrasonic sensor)	\$2.00
12	Raspberry pi Camera Module	\$20
13	PI 0 W camera cable	\$10
14	Miscellaneous Hardware (nuts, bolts, spacers)	\$5.00
15	Laptop (for programming)	\$800.00

Table 5: List of Components and Cost of each.

Note:

- The laptop cost applies only to initial development and is not a recurring material expense.
- The total hardware cost (excluding laptop) for the prototype is approximately 131 CAD.

Total Cost: \$160/- only

6. Software Requirements Specification (SRS)

Project:

Document: Software Requirements Specification

Author: Bilal Hussain

Date: December 5, 2025

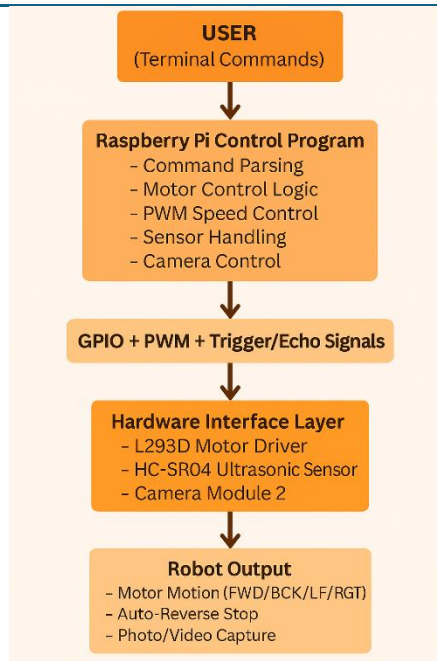
Version: 2

Overall Description

Background: There is an increasing need for low-cost robotic systems that demonstrate how sensing, imaging, and motion control work together in real time. Educational robotics platforms benefit greatly from combining motors, sensors, and cameras to teach students how robots perceive and react to their environment.

Project Overview: In Phase 2, the program extends the robot's capabilities by adding ultrasonic sensing for obstacle detection and a camera module for capturing photos and videos. The Raspberry Pi now reads user commands, monitors distance continuously, and uses GPIO, PWM, and libcamera tools to perform safe movement, automatic reverse stopping, and media recording—all integrated into a single interactive control system.

Block Diagram:



RoboRide Report

Overall Description

Block Descriptions: The block diagram provides a high-level representation of how the RoboRide Phase-2 system processes user commands and converts them into robot actions. The flow begins with the user entering commands through the terminal. These commands are interpreted by the Raspberry Pi control program, which handles motor logic, PWM speed control, ultrasonic sensing, and camera functions. The processed signals are then output as GPIO and PWM values to the hardware interface layer, where the L293D driver, ultrasonic sensor, and camera module operate. Finally, the robot acts on these signals to produce motion, safety responses, and media capture. This structured flow illustrates the clear separation between user interaction, software decision-making, and hardware execution.

Typical User: The typical user is a student, hobbyist, or beginner in embedded systems who wants to learn how real-time sensing, imaging, and motor control work together in a robotic platform. They may have basic programming or Raspberry Pi experience, but no advanced robotics knowledge is required. The system is designed so the user can operate the robot through simple terminal commands while observing movement, live distance readings, and camera feedback in real time.

Constraints:

- The program must run on a Raspberry Pi Zero W with limited GPIO availability, processing speed, and memory.
- The system depends on WiringPi, softPwm, ncurses, and libcamera, all of which must be supported by the installed OS.
- Control is limited to a terminal interface; no mobile app, GUI, or autonomous navigation beyond reverse obstacle-stop is included.
- Ultrasonic sensing accuracy may vary due to noise, angle, and environmental reflections.
- Power is split between motors, Raspberry Pi, and sensors, making the system sensitive to battery health and voltage drops.
- Physical wiring space is limited, especially with the addition of the ultrasonic sensor and camera, increasing the chance of loose connections.
- Motors must be stopped before reversing direction, and reverse motion must halt automatically when an obstacle is detected to ensure hardware safety.

RoboRide Report

Specific Requirements

External Interfaces:

1) User Input Interface:

Keyboard-controlled terminal interface where the user enters commands such as:

- **Movement:** W (Forward), S (Backward), A (Left), D (Right), SPACE (Stop)
- **Speed Control:** 1 (Slow), 2 (Fast)
- **Camera Functions:** C (Capture Photo), V (Start Video), B (Stop Video)
- **Program Control:** Q (Exit)

The interface supports real-time, non-blocking key presses for smooth robot operation.

2) Hardware Output Interface:

The Raspberry Pi outputs control signals through multiple hardware channels:

- **GPIO Digital Signals:** Sent to the L293D motor driver to set wheel direction (forward, backward, turn).
- **PWM Signals:** Generated using softPwm to regulate motor speed.
- **Ultrasonic Trigger Pulses:** GPIO sends timed pulses to the HC-SR04 TRIG pin for distance measurement.
- **Camera Commands:** libcamera-still and libcamera-vid tools capture images and video based on software requests.

3) Display / Feedback Interface:

The ncurses-based terminal UI provides real-time feedback including:

- **Motion Status:** "FORWARD", "BACKWARD", "TURN LEFT", "STOPPED", etc.
- **Speed Mode:** "SLOW" or "FAST"
- **Camera Status:** "Photo saved", "Recording started", "Recording stopped"
- **Sensor Readings:** Live distance values and obstacle alerts (e.g., "Reverse Obstacle: TOO CLOSE!")

This feedback ensures the user always knows the robot's current state and safety conditions.

Specific Requirements:

Below listed are all requirements that exist for this program and given each a MuSCoW rating.

RoboRide Report

Number	Requirement	MuSCoW
1	Platform Requirements The software must run on a Raspberry Pi Zero W using Raspberry Pi OS Lite. Development can be done on Windows/Linux/macOS, but execution must occur on the Pi.	Must
2	Interface Requirements The system uses an interactive ncurses-based terminal interface with real-time keyboard input. No GUI is required. Supports camera and sensor feedback on-screen.	Must
3	User I/P requirements Users can input valid commands only: W, A, S, D (movement), SPACE (stop), 1/2 (speed), C (photo), V (start video), B (stop video), Q (quit). Inputs outside this set must be ignored or return error messages.	Must
4	Calculation Requirements Must compute ultrasonic distance using microsecond timing with acceptable precision ($\pm 1-2$ cm). Motor PWM must run within 0–100% duty cycle. Should handle timing for video recording and timestamp generation.	Should
5	Visual O/P Requirements The terminal must display movement state, speed mode, sensor distance, obstacle warnings, and camera status clearly. Data should refresh smoothly without flicker.	Should
6	Storage/Control O/P Requirements Photos and videos should be stored locally in ~/Desktop, and the system must correctly control hardware: motors through L293D, ultrasonic sensor via TRIG/ECHO, and camera via libcamera.	Could
7	Documentation Requirements Clear documentation must be provided, including: setup guide, wiring diagrams, required libraries, execution steps, and user instructions for movement, sensor reading, and camera use.	Must

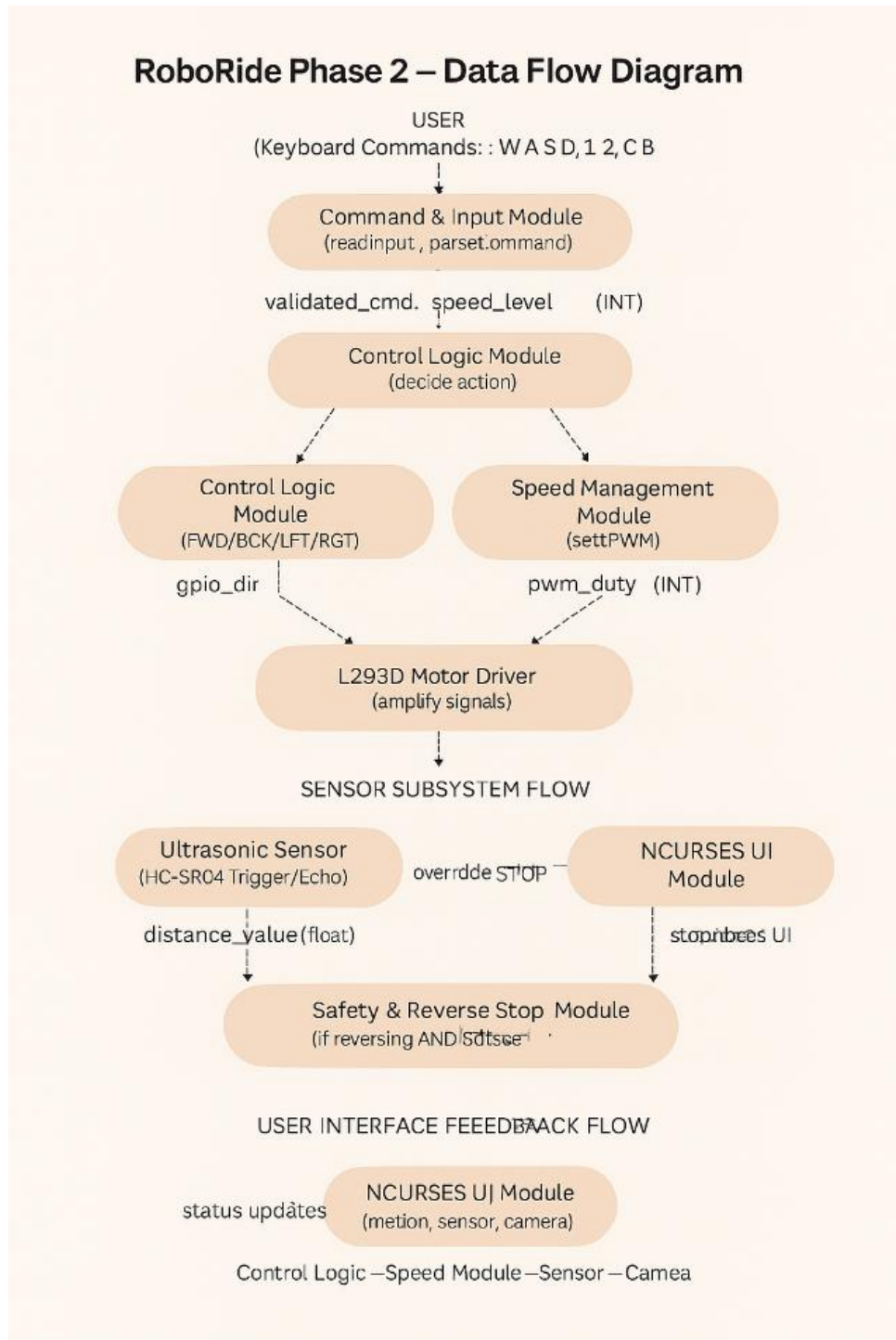
7. Software Design Description (SDD)

Project:	
Document:	Software Design Document
Author:	Bilal Hussain
Date:	December 05, 2025
Version:	2

System Description	
System Description:	The RoboRide software controls a two-wheel robotic vehicle using a Raspberry Pi Zero W. It processes real-time keyboard input and translates it into motor actions such as forward, backward, turning, speed selection, and emergency stopping. In Phase 2, the system was expanded with an HC-SR04 ultrasonic sensor for continuous distance monitoring and an automatic reverse-stop safety feature. The Raspberry Pi Camera Module 2 was also integrated, enabling photo capture and video recording directly from the robot. The ncurses-based interface displays live sensor data, motion status, and camera activity, creating a fully interactive control environment.
Development Platform:	The software was developed using Raspberry Pi OS Lite with the GNU GCC compiler and the C programming language. The system uses the WiringPi and softPwm libraries to implement GPIO control and PWM-based motor speed regulation, while ncurses is used for building a text-based user interface. For Phase 2 additions, libcamera tools were incorporated to support still-image capture and video recording. Development and debugging were performed either directly on the Raspberry Pi or via SSH/VNC using a separate computer.
Execution Platform:	The program runs natively on the Raspberry Pi Zero W, which serves as the central controller for all hardware components. The Pi sends PWM and logic signals to the L293D motor driver to control the DC motors and reads distance measurements from the HC-SR04 ultrasonic sensor through GPIO pins. It also triggers the camera module for capturing photos and recording videos. The execution platform enables simultaneous motion control, obstacle detection, and camera operation—allowing RoboRide to function as a motorized, sensor-aware, multimedia-capable robotic system.

RoboRide Report

Data Flow Diagram:



RoboRide Report

System Interfaces:

Input (Terminal prompt):

```
=====
                        RoboRide
                        Drive and Capture
=====

Controls:
  W = Forward      S = Backward
  A = Left         D = Right
  1 = Slow Speed   2 = Fast Speed
  SPACE = Stop     Q = Quit

Camera:
  C = Capture Photo
  V = Start Video Recording
  B = Stop Video Recording

Status:
  Motion: STOPPED
  Speed Mode: SLOW
Camera: Ready
Camera: (no video recording in progress)

Sensors:
  Distance: --- (no echo)
  Reverse Obstacle: UNKNOWN
```

Normal output (status/feedback):

```
=====
                        RoboRide
                        Drive and Capture
=====

Controls:
  W = Forward      S = Backward
  A = Left         D = Right
  1 = Slow Speed   2 = Fast Speed
  SPACE = Stop     Q = Quit

Camera:
  C = Capture Photo
  V = Start Video Recording
  B = Stop Video Recording

Status:
  Motion: TURN RIGHT
  Speed Mode: SLOW
Camera: Ready
Camera: (no video recording in progress)

Sensors:
  Distance: --- (no echo)
  Reverse Obstacle: UNKNOWN |
```

RoboRide Report

Error messages (invalid/unsafe input):

```
=====
                        RoboRide
                      Drive and Capture
=====

Controls:
  W = Forward      S = Backward
  A = Left         D = Right
  1 = Slow Speed   2 = Fast Speed
  SPACE = Stop     Q = Quit

Camera:
  C = Capture Photo
  V = Start Video Recording
  B = Stop Video Recording

Status:
  Motion: TURN RIGHT
  Speed Mode: SLOW
Camera: Ready
Camera: (no video recording in progress)

Sensors:
  Distance: --- (no echo)
  Reverse Obstacle: UNKNOWN |
```

Note: We made the UI wall strong so it does not react to garbage inputs.

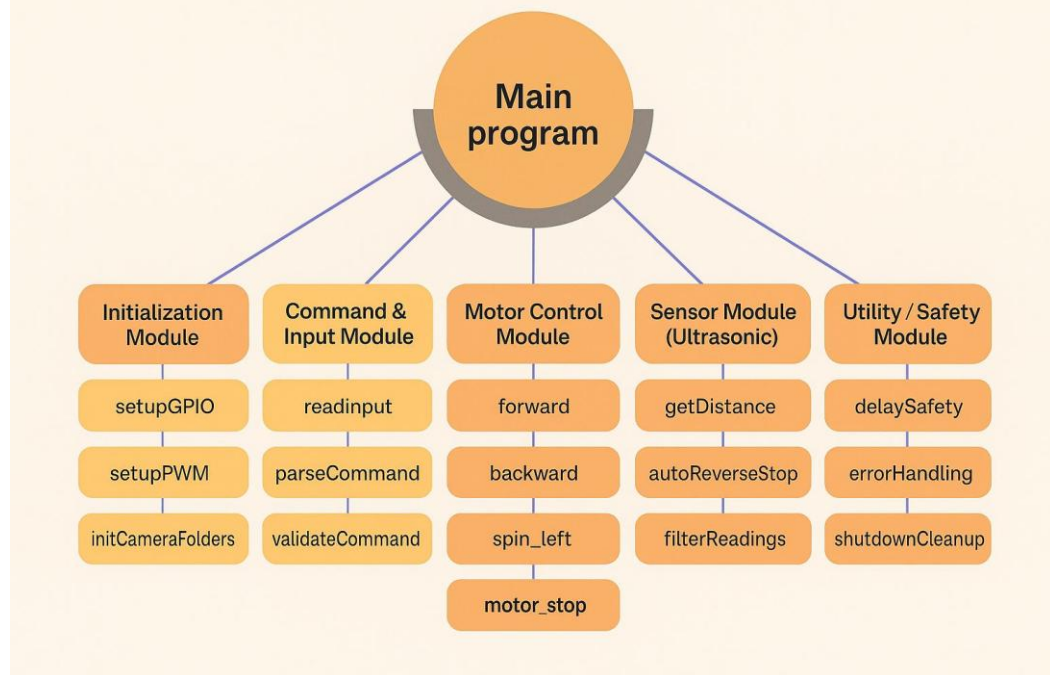
Program exit:

```
taha@raspberrypi:~ $ ./carcook
taha@raspberrypi:~ $ |
```

Program exits safely.

Detailed Design

Hierarchy Diagram:



Modules (one for each block shown in the hierarchy diagram)

Main Program

This is the central controller that coordinates all modules. It initializes the system, reads user commands, triggers the correct actions, updates the UI, and manages real-time motor, sensor, and camera operations.

Initialization Module

setupGPIO / setupPWM / initCameraFolders

This module prepares all hardware and software resources. It configures GPIO pins, starts PWM channels, and creates folders for storing captured images and videos.

Command & Input Module

readInput / parseCommand / validateCommand

This module handles keyboard input from the terminal. It reads user commands, interprets them, and ensures they are valid before passing them to the control logic.

Motor Control Module

forward / backward / spin_left / spin_right / motor_stop

This module executes all motor-related actions. Based on user commands, it sets the motor directions and applies PWM values to control speed and motion.

Sensor Module (Ultrasonic)

getDistance / autoReverseStop / filterReadings

This module measures distance using the HC-SR04 sensor. It filters noisy readings and automatically stops the robot during reverse if an obstacle is detected within the safety threshold.

RoboRide Report

Camera Module

capture_photo / start_video / stop_video

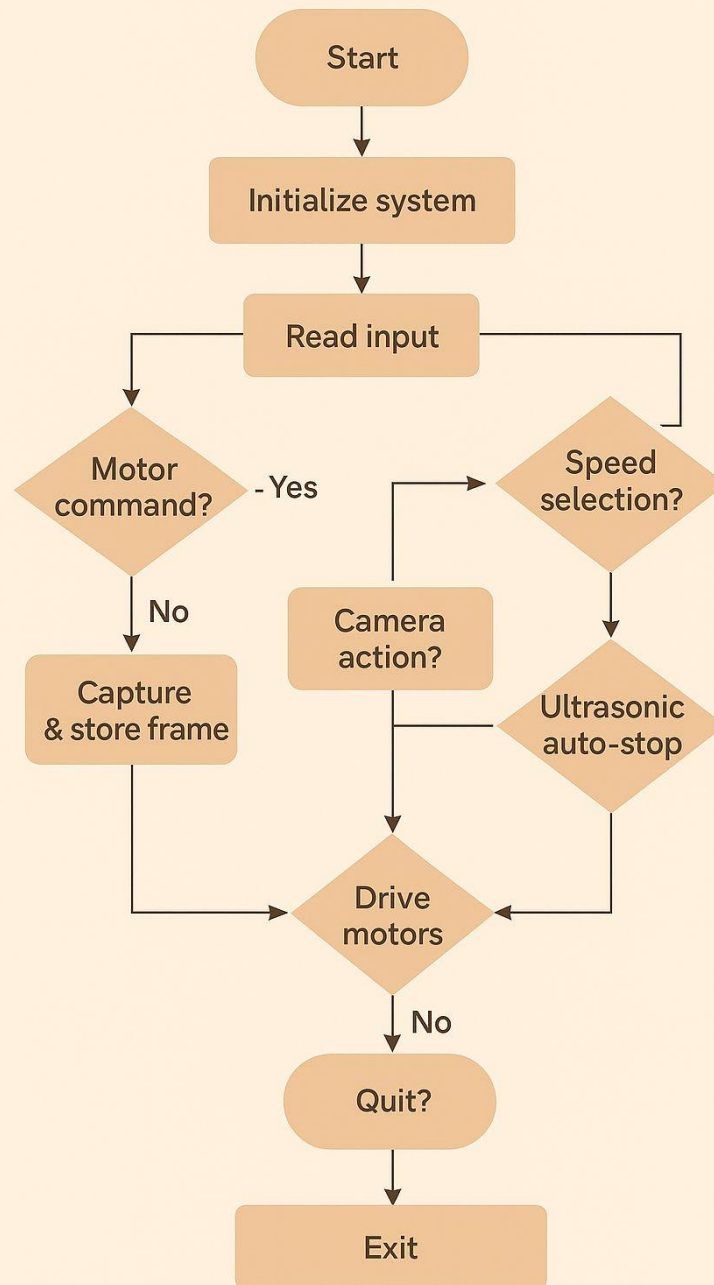
Handles all camera operations. It pauses the robot safely before taking photos or videos, generates timestamped filenames, and stores media in organized directories.

Utility / Safety Module

delaySafety / errorHandling / shutdown Cleanup

This module contains safety routines that prevent abrupt motor direction changes, handle unexpected errors, and ensure motors and camera processes stop cleanly on exit.

Flow chart:



RoboRide Report

Test Values

Reliability

Test:

- **Input:** Reverse at slow speed (70%)
Expected Output: Robot moves backward smoothly and stops automatically when an obstacle is < 20 cm.
- **Input:** Reverse at fast speed (90%)
Expected Output: Robot moves backward faster but still performs a safe auto-stop before collision.
- **Input:** Take 5 photos + Start/Stop 2 video recordings
Expected Output: All images and videos are saved correctly, and the robot pauses safely during capture.
- **Input:** Read distance every 2 seconds for 30 minutes
Expected Output: Program runs continuously without freezing; distance readings show stable values ($\pm 2-3$ cm).

Robustness

Test:

- **Input:** Invalid command (e.g., XYZ)
Expected Output: “last command” message is shown; robot does not move.
- **Input:** Mixed-case command (e.g., bAcKwArD)
Expected Output: Command executes correctly (case-insensitive).
- **Input:** Rapid alternating commands (e.g., $W \rightarrow S \rightarrow D$ quickly)
Expected Output: Program always performs a `motor_stop()` before switching direction, preventing damage.
- **Input:** Sensor noise (hand waved quickly near ultrasonic sensor)
Expected Output: Distance shows occasional “no echo,” but system remains stable and avoids false auto-stops.

Exceptions

Test:

- **Exception:** Ultrasonic sensor fails (no echo for > 250 ms)
Handling: Program displays “--- (no echo)” and continues safely without stopping the robot unnecessarily.
- **Exception:** User starts camera recording while robot is moving
Handling: Program automatically calls `motor_stop()` before taking photo or video.
- **Exception:** Video process still running when user presses STOP VIDEO i.e. B
Handling: `kill libcamera-vid` safely terminates recording and updates UI.
- **Exception:** Distance reading is out of valid range (fabric, angled surface, etc.)
Handling: Reading is discarded; displayed as “no echo” and robot ignores it.
- **Exception:** Program is closed while motors were running
Handling: `motor_stop()` is called before exit (safe shutdown).

8.2 Schematic Diagram

Schematic diagram of the RoboRide circuit showing connections between the Raspberry Pi, Pi camera, HC-SR04 Ultrasonic sensor, L293D motor driver, DC motors, and dual power supplies.

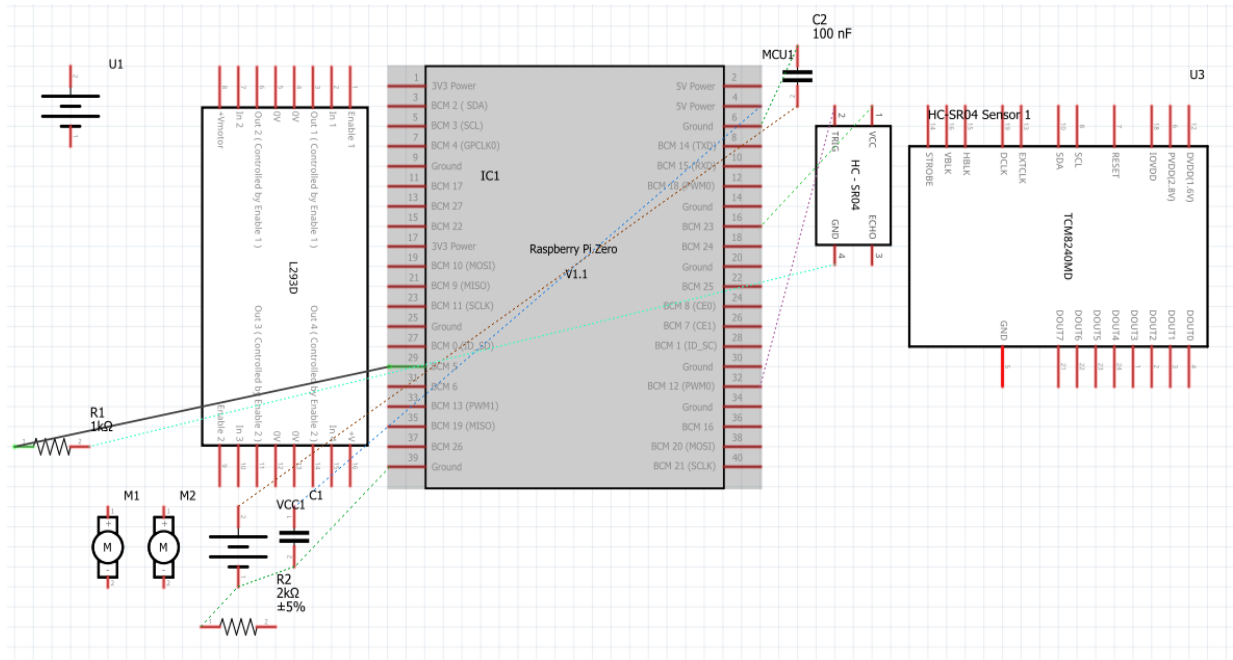


Figure 10: RoboRide wiring schematic illustrating power, control, and ground connections.

8.4 Wiring Description

- The following table lists all wiring connections between the Raspberry Pi, L293D motor driver, HC-SR04, Camera, power supply, and motors.

No.	Component / Pin	Connected To	Component Pin	Description
1	Raspberry Pi GPIO 17 (WiringPi 0)	L293D Pin 2 (IN1)	Digital Output	Controls Motor A direction 1.
2	Raspberry Pi GPIO 18 (WiringPi 1)	L293D Pin 7 (IN2)	Digital Output	Controls Motor A direction 2.
3	Raspberry Pi GPIO 27 (WiringPi 2)	L293D Pin 1 (ENA)	PWM Output	Enables Motor A speed control via PWM.
4	Raspberry Pi GPIO 22 (WiringPi 3)	L293D Pin 10 (IN3)	Digital Output	Controls Motor B direction 1.
5	Raspberry Pi GPIO 23 (WiringPi 4)	L293D Pin 15 (IN4)	Digital Output	Controls Motor B direction 2.
6	Raspberry Pi GPIO 24 (WiringPi 5)	L293D Pin 9 (ENB)	PWM Output	Enables Motor B speed control via PWM.
7	L293D Pin 3 & 6	DC Motor A	Motor Output	Connects to left motor terminals.
8	L293D Pin 11 & 14	DC Motor B	Motor Output	Connects to right motor terminals.
9	L293D Pin 8 (Vcc2)	4 × AA Battery Pack ($\approx 6\text{ V} +$)	Power Input	Provides motor power supply.
10	L293D Pin 16 (Vcc1)	Raspberry Pi 5 V (Pin 2 or 4)	Logic Power	Supplies logic voltage to driver IC.
11	L293D Ground Pins (4, 5, 12, 13)	Common Ground (Pi GND + Battery $-$)	Ground	Completes circuit return path.
12	Raspberry Pi GND (Pin 6 or 9)	Breadboard Ground Rail	Ground	Shared ground for stable reference.
13	Breadboard Power Rail	Power Bank 5 V Output	Power	Provides regulated 5 V supply to Pi.
14	Ultrasonic TRIG – Raspberry Pi GPIO 5 (WiringPi 21)	HC-SR04 TRIG Pin	Digital Output	Sends 10 μs trigger pulse to start measurement.
15	Ultrasonic ECHO → Voltage Divider → Raspberry Pi GPIO 12 (WiringPi 26)	HC-SR04 ECHO Pin	Digital Input	Reads echo pulse (5 V → 3.3 V using 2k Ω + 1k Ω divider).

RoboRide Report

16	HC-SR04 Vcc	Breadboard 5 V Rail	Power	Provides sensor operating voltage.
17	HC-SR04 GND	Breadboard Ground	Ground	Sensor ground reference.
18	Pi Zero Camera Connector (CSI)	Camera Module 2	—	Wide FFC cable for video/photo capture

Table 6: Wiring Help Table for RoboRide.

9. Implementation

9.1 Code Structure

The *RoboRide* program was written in C language and executed on a Raspberry Pi using the WiringPi, Ncurses and softPwm libraries. The overall structure of the code follows a modular and logical flow that enables clear control of motor direction and speed. The major sections of the code are outlined below:

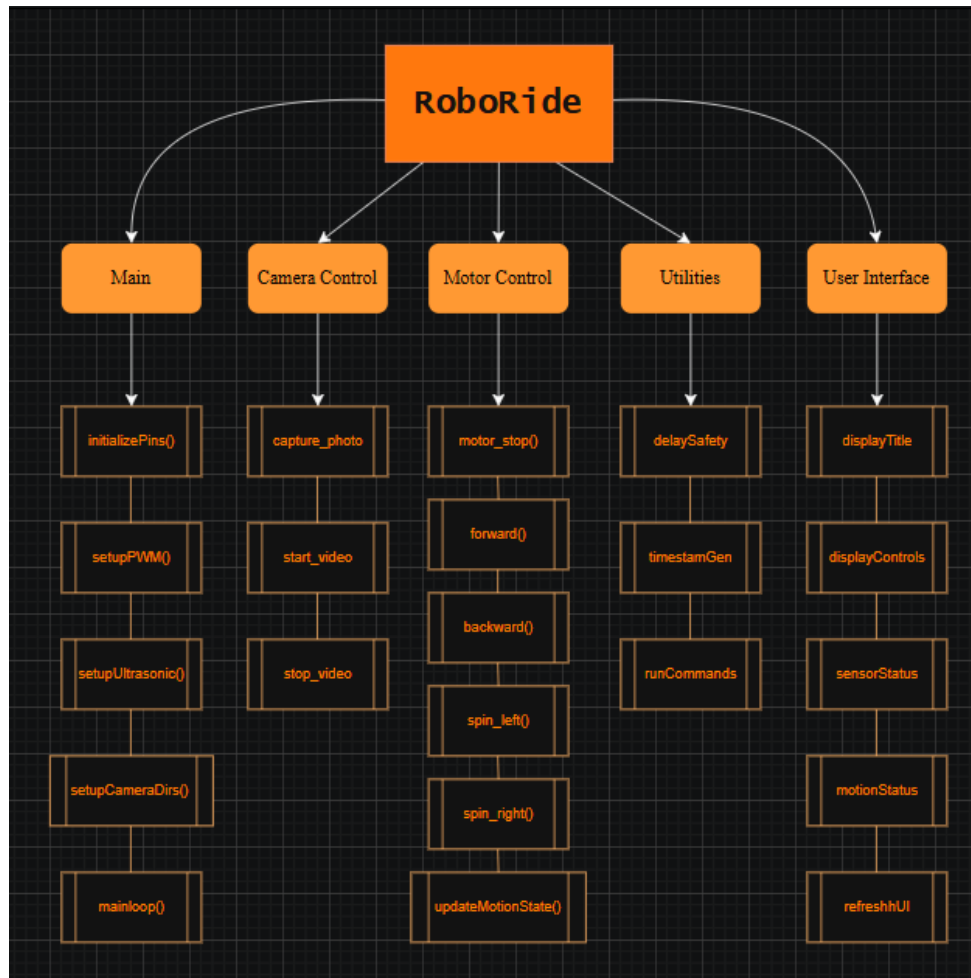


Figure 11: Code Structure Diagram of the RoboRide System created using Draw.io. (draw.io, n.d.)

9.2 Key Code Snippets

1. Motor Stop Function

```
60 void motor_stop(void) {
61     digitalWrite(IN1, LOW);
62     digitalWrite(IN2, LOW);
63     digitalWrite(IN3, LOW);
64     digitalWrite(IN4, LOW);
65     softPwmWrite(ENA, 0);
66     softPwmWrite(ENB, 0);
67     delay(100); // small safety delay
68 }
```

Explanation:

This function immediately stops both motors by disabling all motor driver inputs and setting PWM to 0.

The 100 ms delay provides electrical and mechanical protection by preventing abrupt direction switching.

2. Forward Motor Control

```
70 void forward(int speed) {
71     motor_stop();
72     digitalWrite(IN1, HIGH);
73     digitalWrite(IN2, LOW);
74     digitalWrite(IN3, HIGH);
75     digitalWrite(IN4, LOW);
76     softPwmWrite(ENA, speed);
77     softPwmWrite(ENB, speed);
78 }
```

Explanation:

This defines the motor direction signals required to move the robot forward.

The function first calls `motor_stop()` to prevent sudden direction changes and then applies PWM-based speed control.

3. Ultrasonic Distance Measurement

```
112 float getDistance(void) {
113     // Ensure TRIG low
114     digitalWrite(TRIG, LOW);
115     delayMicroseconds(2);
116
117     // 10 µs trigger pulse
118     digitalWrite(TRIG, HIGH);
119     delayMicroseconds(10);
120     digitalWrite(TRIG, LOW);
121
122     // Wait for echo start
123     unsigned long start = micros();
124     while (digitalRead(ECHO) == LOW) {
125         if (micros() - start > 250000) { // 250 ms timeout
126             return -1.0f;
127         }
128     }
129
130     unsigned long echo_start = micros();
131
132     // Wait for echo end
133     while (digitalRead(ECHO) == HIGH) {
134         if (micros() - echo_start > 250000) { // 250 ms timeout
135             return -1.0f;
136         }
137     }
138
139     unsigned long echo_end = micros();
140     float distance = (echo_end - echo_start) / 58.0f; // HC-SR04 formula (cm)
141
142     if (distance < 2.0f || distance > 400.0f) {
143         return -1.0f;
144     }
145
146     return distance;
147 }
```

Explanation:

This section sends a 10 µs trigger pulse and waits for the ultrasonic echo signal to rise. A timeout is included to avoid system hangs if the sensor fails or no signal is returned.

4. Converting Echo Time to Distance

```
139     unsigned long echo_end = micros();
140     float distance = (echo_end - echo_start) / 58.0f; // HC-SR04 formula (cm)
```

Explanation:

The HC-SR04 sensor's return time is converted into distance using the standard formula based on the speed of sound.

Dividing by 58 converts microseconds into centimeters.

5. Safety Auto Stop While Reversing

```
381 // Auto-stop only when reversing and obstacle is too close
382 if (motion_state == BACKWARD_M && d < REVERSE_STOP_DISTANCE_CM) {
383     motor_stop();
384     motion_state = STOPPED;
385     mvprintw(16, 0, " Motion: STOPPED (AUTO REVERSE STOP) ");
386     mvprintw(23, 0, " Reverse Obstacle: TOO CLOSE! ");
387 } else {
388     mvprintw(23, 0, " Reverse Obstacle: CLEAR ");
389 }
390 }
391 }
```

Explanation:

While reversing, if the ultrasonic sensor detects an object closer than 50 cm, the robot automatically stops.

This prevents collisions and adds basic safety and autonomy to the vehicle.

6. Capture a Photo

```
181 void capture_photo(void) {
182     motor_stop();
183     motion_state = STOPPED;
184
185     ensure_media_dirs();
186
187     char path[512];
188     make_timestamp_path(path, sizeof(path), "Pictures", "photo", "jpg");
189
190     char cmd[768];
191     // -n = no preview (works over SSH), redirect output to /dev/null
192     snprintf(cmd, sizeof(cmd),
193             "libcamera-still -n -o '%s' >/dev/null 2>&1",
194             path);
195
196     mvprintw(18, 0, "Camera: Capturing photo... ");
197     refresh();
198     system(cmd);
199 }
```

Explanation:

Before capturing a photo, the robot stops to ensure a stable image and safe operation. The command is executed without preview using libcamera-still, and the image is saved with a timestamp.

7. Start Continuous Video Recording

```
214     snprintf(cmd, sizeof(cmd),
215              "libcamera-vid -n -t 0 -o '%s' >/dev/null 2>&1 &",
216              path);
217
218     mvprintw(19, 0, "Camera: Starting video recording...");
219     refresh();
220     system(cmd);
```

Explanation:

This starts an indefinite video recording running in the background.

-t 0 means “record until manually stopped,” and & makes the process run without blocking robot controls.

8. Ncurses based Setup

```
259     // Ncurses setup
260     initscr();
261     cbreak();
262     noecho();
263     nodelay(stdscr, TRUE); // getch() non-blocking
264     keypad(stdscr, TRUE);
```

Explanation:

These commands initialize the ncurses interface for real-time keyboard control.

nodelay() makes keypress reading non-blocking so the robot can drive and sense at the same time.

9. Keyboard Input Handling

```
000      ch = getch();
001      if (ch != ERR) {
002          switch (ch) {
003              case 'w': case 'W':
004                  forward(speed);
005                  motion_state = FORWARD_M;
006                  mvprintw(16, 0, " Motion: FORWARD");
007                  break;
008
009              case 's': case 'S':
010                  backward(speed);
011                  motion_state = BACKWARD_M;
012                  mvprintw(16, 0, " Motion: BACKWARD");
013                  break;
014          }
```

Explanation:

This block checks for user keypresses and maps them directly to movement or camera actions.

The non-blocking behavior ensures the robot remains responsive and updates the UI instantly.

10. Periodic Ultrasonic Sampling

```
369      if (now - last_dist_ms > 2000) {
370          last_dist_ms = now;
371
372          float d = getDistance();
```

Explanation:

The ultrasonic sensor is read every 2 seconds to avoid excessive noise and to keep processing smooth.

This timed sampling works in parallel with movement and camera features.

9.4 Challenges and Solutions

Challenge	Description	Solution Implemented
Motor startup hesitation	Motors sometimes required a small manual push to start due to low initial torque.	Adjusted PWM values (70% for slow, 90% for fast) and ensured both motors stop before direction changes.
Pin congestion on Raspberry Pi	Limited GPIO space and tight physical wiring caused confusion during setup.	Created a clear pin map and verified each connection with a multimeter and Fritzing schematic.
Polarity issues	Robot initially moved in reverse when “forward” was commanded.	Swapped the polarity of one motor’s connection on the L293D output pins.
Connection to Pi	Loose jumper wires occasionally caused motor flickering.	Used tighter male-female jumper connectors and secured the breadboard connections.
Pi Connection	Had problems like pi was not connecting to hotspot.	Enabled 2.4Ghz hotspot connection.
Motor Drift / Uneven Movement	Motor Drift / Uneven Movement	Motor Drift / Uneven Movement
Ultrasonic Sensor Not Detecting Echo	Ultrasonic Sensor Not Detecting Echo	Ultrasonic Sensor Not Detecting Echo
Camera Not Detected on Pi Zero W	Raspberry Pi Camera Module 2 initially failed to initialize due to incorrect cable orientation and disabled camera interface.	Flipped FFC cable correctly, enabled camera support in raspi-config, and verified with libcamera-still.
Car Did Not Stop While Capturing Photos/Videos	Running camera commands while robot was moving caused unsafe motion during photography.	Modified program logic to automatically stop motors during any photo/video capture process.
System Instability From Tight Wiring	The compact chassis caused tangled wires, accidental disconnects, and mechanical strain during movement.	Reorganized wiring using shorter jumpers, taped weak joints, and repositioned power bank + sensors to reduce tension.

Table 7: Challenges and Solutions explained using a table.

10. Testing and Results

10.1 Testing Plan

- **T1 – Forward Movement Test**

This test verified the robot's ability to move straight ahead when the “W” command was entered. Both DC motors rotated in the forward direction, powered through the L293D motor driver. The robot maintained a straight trajectory with minimal deviation, confirming correct motor wiring and signal output from the Raspberry Pi. (Fig 12)

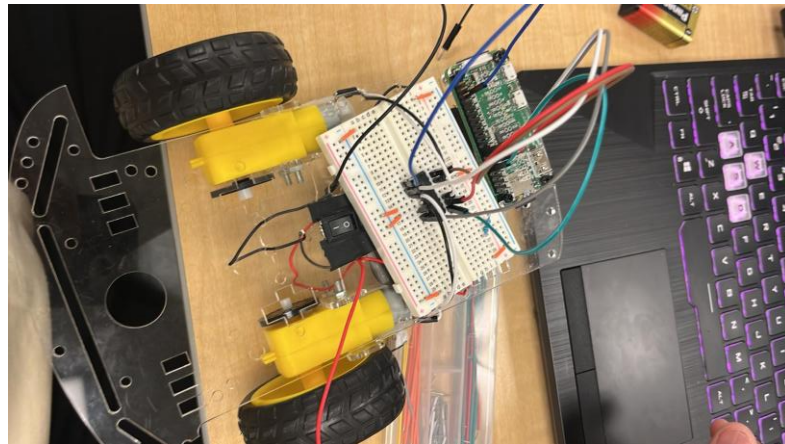


Figure 12: Forward Movement Test. Video link : [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T2 – Backward Movement Test**

The purpose of this test was to validate the reverse functionality. When the “S” command was given, both motors reversed direction. The motion was smooth and stable, confirming proper polarity configuration and successful execution of reverse logic in the program. (Fig 13)

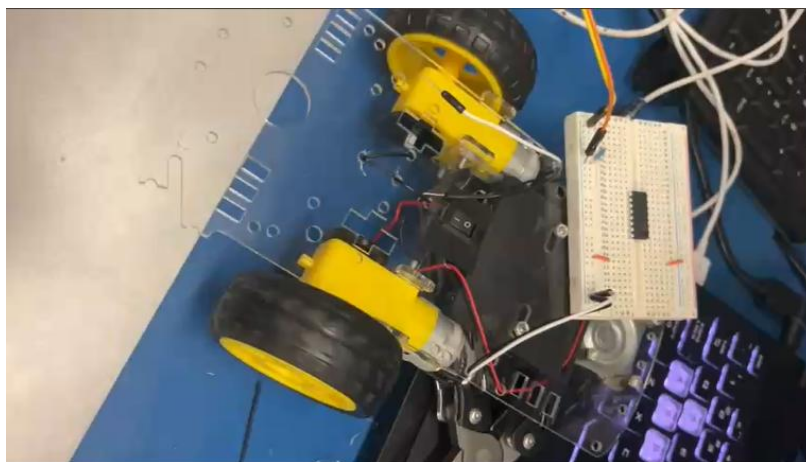


Figure 13: Backward Movement Test. Video link : [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T3 – Left Turn Test**

This test ensured that the robot could rotate left on its axis. The right motor rotated forward while the left motor rotated backward. The robot performed an in-place left turn, demonstrating accurate direction control through the L293D's differential drive system. (Fig 14)

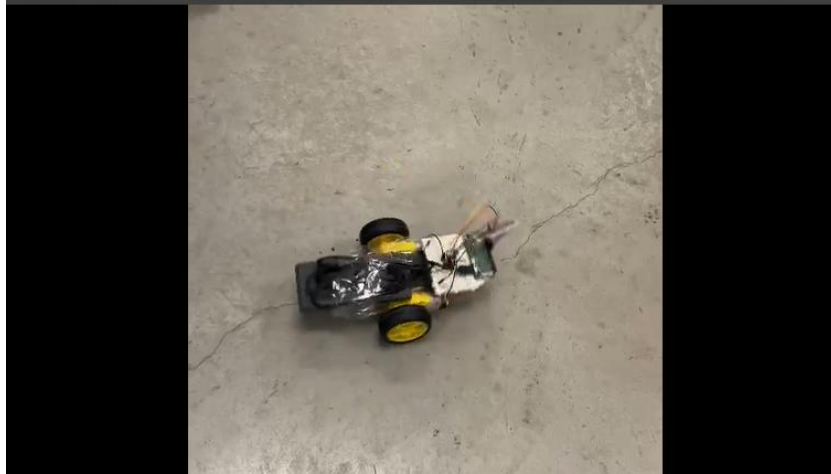


Figure 14: Left Turn Test. [Video link : Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T4 – Right Turn Test**

The “D” command was tested for correct right rotation. The left motor rotated forward, and the right motor rotated backward, producing a clean spin to the right. The motion confirmed correct GPIO mapping and direction control signals. (Fig 15)

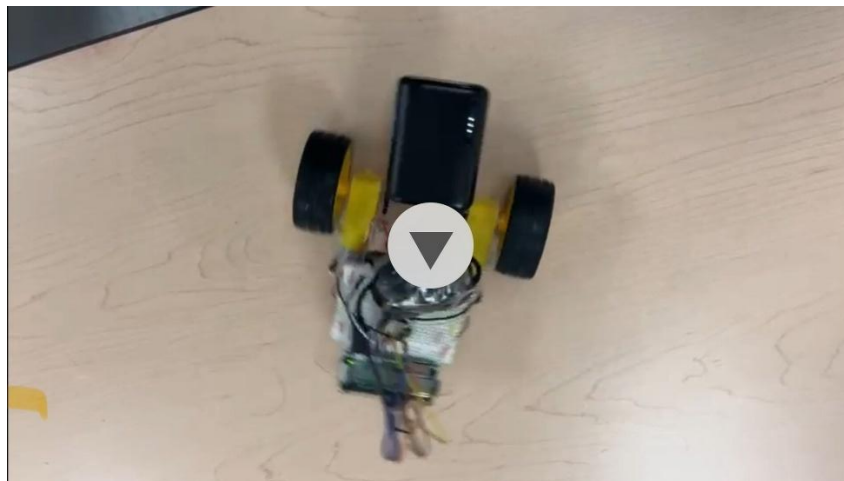


Figure 15: Backward Movement Test. [Video link : Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T5 – Speed control test through duty cycle**

Tested PWM-based speed control at 70% (slow) and 90% (fast) duty cycles. Oscilloscope confirmed clean PWM signals with accurate duty variation. Robot speed changed smoothly between modes, verifying effective speed modulation. (Fig 16)



Figure 16: Duty Cycle Test. **Video link :** [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T6 – Oscilloscope PWM Test**

An oscilloscope was connected to the ENA and ENB pins to observe PWM waveforms. Clean square waves were displayed with duty cycles accurately reflecting the programmed values. The test verified that PWM signals were stable, consistent, and free of electrical noise. (Fig 17)

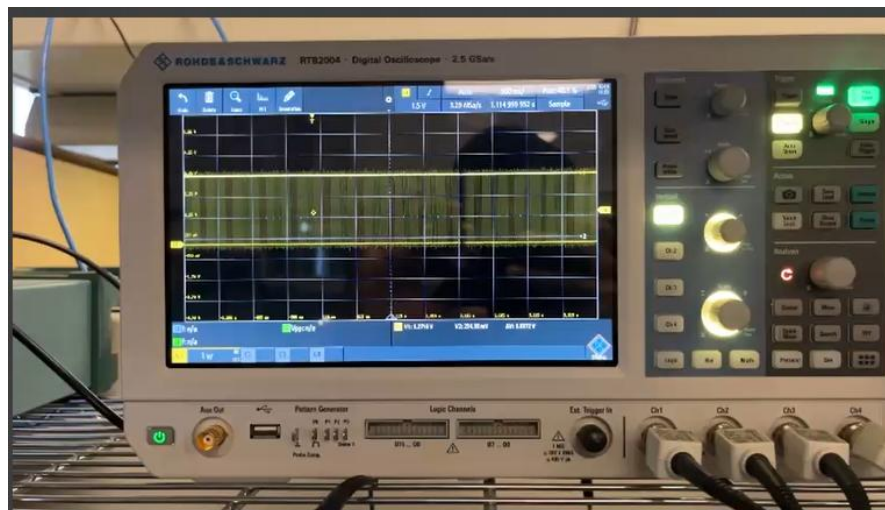


Figure 17: Oscilloscope PWM Test. **Video link :** [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T7 – Motor Stop Test**

This test checked the response to the “STOP” command. Both motors halted immediately upon receiving the command, with PWM duty reduced to zero. This verified that the braking mechanism and safety delay functions operated as intended. (fig 18)



Figure 18: Motor Stop Test. **Video link :** [Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](https://www.youtube.com/watch?v=Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1)

- **T8 – Power Interruption Test**

```
=====
                        RoboRide
                      Drive and Capture
=====

Controls:
W = Forward      S = Backward
A = Left         D = Right
1 = Slow Speed   2 = Fast Speed
SPACE = Stop     Q = Quit

Camera:
C = Capture Photo
V = Start Video Recording
B = Stop Video Recording

Status:
Motion: TURN RIGHT
Speed Mode: SLOW
Camera: Ready
Camera: (no video recording in progress)

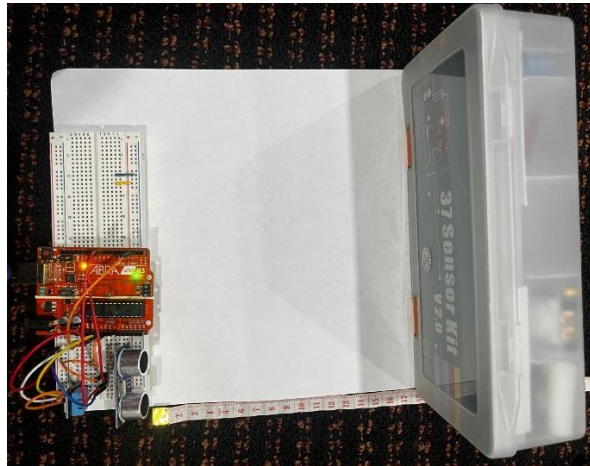
Sensors:
Distance: --- (no echo)
Reverse Obstacle: UNKNOWN |
```

Figure 19: Picture from the system when power is disconnected.

In this test, the system was powered off mid-operation and then restarted. Upon reboot, the robot froze in proper function without configuration loss. The result demonstrated reliable hardware initialization and stable software re-execution after power cycling.

RoboRide Report

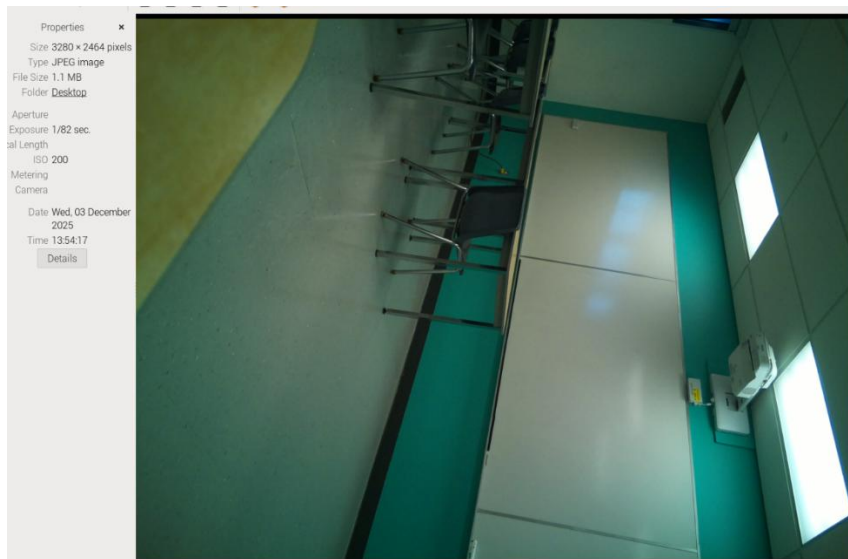
- **T9 – Ultrasonic sensor in different Temperatures with accuracy test.**



In this test Ultrasonic sensor readings were taken and compared in 3 different temperatures with the help of a measuring tape by the setup to compare real time reading to actual readings. Room (25°), Low (0°) and High (40°). The testing was successful; sensor gave similar reading in all temperatures. Error % was negligible i.e. 0.2°-0.5°.

Note: More pictures from this test can be found in Appendices in this report.

- **T10 – Camera Capture Test**



In this test, camera was connected to the pi and images were captured. The images were set to be saved through a path in the photos folder in the raspberry pi. VNC viewer was used to view and verify the pictures. Test was successful and all the criteria's were met.

RoboRide Report

- **T11 – Final RoboRide test**

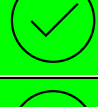
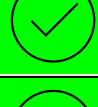
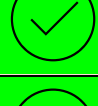
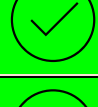
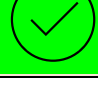
During this test, before the final demonstration, we made sure all the sensors are working perfectly and batteries were charged fully so that RoboRide is working on its full potential. The test was successful as well as the demo.

Note : All Test videos can be found in GitHub. Please follow the link.
[Bilal2866/ENGG-PROJECT-3 at Week-5-Phase-1](#)

RoboRide Report

10.2 Test Results

The testing phase focused on verifying both the software control logic and the hardware performance of the RoboRide system. Each test was conducted on the assembled prototype powered by the Raspberry Pi Zero W and L293D motor driver.

Case ID	Test Description	Expected Result	Actual Result	Status
T1	Verify forward movement of both motors when “FWD” command is entered.	Both motors rotate forward; robot moves straight ahead.	Robot moves straight with minimal deviation.	
T2	Verify backward movement using “BCK” command.	Both motors rotate in reverse; robot moves backward smoothly.	Functioning as expected.	
T3	Test left rotation using “LFT” command.	Right motor rotates forward; left motor rotates backward, causing left spin.	Achieved correct turning motion.	
T4	Test right rotation using “RGT” command.	Left motor rotates forward; right motor rotates backward, causing right spin.	Correct rotation observed.	
T5	Test speed control between slow (70%) and fast (90%) settings.	Motor speed adjusts smoothly with visible difference between modes.	Smooth transition observed.	
T6	Oscilloscope PWM test for ENA and ENB pins.	PWM signal frequency and duty cycle correspond to set speed percentages.	Verified clean square wave with accurate duty modulation on oscilloscope.	
T7	Test emergency stop (“STOP” command).	Motors stop immediately; no further motion.	Works as expected.	
T8	Power interruption test.	Robot resumes correct operation after power restoration.	Successfully resumes after restart.	
T9	HC-SR04 Ultrasonic test at different Temperatures and accuracy Test	Correct distance readings in different temperatures.	The accuracy was good in different environments.	
T10	Camera Capture Test	Camera must take pictures and videos on command and save it,	Camera captures and saved successfully.	
T11	Final RoboRide Test	All the sensors must work perfectly and it should stop at <50cm.	RoboRide function met its expectations.	

10.3 Observations

- The robot successfully executed all motion commands — forward, backward, left, a right rotations, capturing on pictures and videos command and obstacle detection without wiring or logic errors.
- PWM output observed on the oscilloscope showed clean, stable square waves on both ENA and ENB pins.
- Duty cycle variation between slow (~70%) and fast (~90%) modes was consistent with programmed values.
- Motor response time was quick, with smooth acceleration and deceleration between speed modes.
- Minor delay at startup was observed due to motor inertia but resolved once movement began.
- The STOP command halted both motors immediately, confirming reliable braking control.
- Power delivery from the 5000 mAh power bank and 6 V battery pack was stable, showing no voltage drop during motion.
- No overheating or short-circuit issues occurred during extended operation.
- The system maintained consistent performance after power interruptions and restarts.
- Ultrasonic sensor gave accurate measurements even in different temperatures, error % was negligible i.e. ~0.5cm.
- Camera captured pictures and videos on command but took about 10s to save.
- Overall, RoboRide demonstrated high reliability, precision in speed control, obstacle detection, capture pictures and videos and smooth maneuverability.

11. Discussion

11.1 Analysis of Results

The RoboRide project successfully achieved all major objectives across both development phases. During Phase 1, the robotic car demonstrated stable and reliable motion control through forward, backward, left, and right movements using PWM-driven speed regulation. The wiring between the Raspberry Pi, L293D motor driver, and DC motors performed consistently, validating both the hardware design and the motor-control logic implemented in C.

In Phase 2, the system was enhanced with sensing and perception capabilities through the integration of the HC-SR04 ultrasonic sensor and the Raspberry Pi Camera Module 2. The ultrasonic sensor provided accurate distance readings and enabled an automatic reverse-stop feature when obstacles were detected below the safety threshold. The camera module functioned reliably, capturing timestamped photos and videos on command while safely pausing robot motion. Overall, the results confirm that RoboRide evolved from a basic motion platform into a more interactive, sensor-aware robotics system.

11.2 Limitations

Despite achieving its objectives, several limitations were observed during testing:

- **Motor torque limitations** — The DC motors struggled under heavier loads and performed best on smooth indoor surfaces.
- **Power constraints** — The AA battery pack and power bank required periodic replacement/charging during extended testing sessions.
- **Ultrasonic sensor blind spots** — The HC-SR04 sensor provided reliable readings only within its ideal range and angle, occasionally producing noise or missed echoes.
- **Camera video storage limitations** — Video recording worked correctly, but storage paths required manual verification because of directory permissions and VNC interactions.
- **Manual control dependency** — Although obstacle detection works during reverse, the robot still relies primarily on manual input and does not navigate autonomously.

11.3 Potential Improvements

To increase system reliability, performance, and real-world usefulness, the following enhancements are recommended:

- **Upgrade to stronger motors** or use a geared motor system to improve torque, weight capacity, and terrain adaptability.
- **Expand sensing** by adding more ultrasonic sensors, infrared sensors, or a time-of-flight sensor for 360° obstacle awareness.
- **Add autonomous navigation features**, such as line following, mapping, or simple AI-based object detection using the camera module.

12. Conclusion

- The RoboRide project successfully evolved from a basic motor-controlled prototype into a more intelligent, sensor-enhanced robotic system. Phase 1 established the foundation by implementing reliable bidirectional motion, turning control, and PWM-based speed regulation using the Raspberry Pi, L293D motor driver, and dual DC motors. Phase 2 expanded these capabilities by integrating the HC-SR04 ultrasonic sensor for obstacle detection and the Raspberry Pi Camera Module 2 for photo and video capture, transforming RoboRide into an interactive and semi-autonomous platform.
- Throughout the development cycle, the team addressed numerous hardware and software challenges, including wiring congestion, unstable motor torque, sensor noise, echo timeouts, and UI clarity. These issues were systematically debugged and resolved through better circuit organization, improved timing logic, safety delays, error handling, and a redesigned ncurses-based control interface. The ability to stop automatically during reverse when obstacles were detected and to pause movement during camera operations significantly improved system robustness and safety.
- Educationally, RoboRide provided an excellent learning experience in embedded systems, robotics, and real-time control. Students gained hands-on experience with PWM, GPIO interfacing, ultrasonic timing, camera drivers, Linux-based programming, and modular C software design. The seamless integration of sensing, mobility, and imaging demonstrated how theoretical concepts translate into a working robotics platform with real-world relevance.

Overall, RoboRide met and exceeded its objectives across both phases, maturing into a versatile robotic system capable of motion control, environment sensing, and multimedia capture. The project lays a strong foundation for future enhancements including wireless control, cloud-connected monitoring, autonomous navigation, AI-based vision, and expanded sensor suites moving RoboRide closer to becoming a fully intelligent mobile robot.

13. References

Datasheets & Technical Specs

- [1] Raspberry Pi Foundation (2021) 'Raspberry Pi 4 Model B Technical Datasheet'. Raspberry Pi Foundation.
- [2] STMicroelectronics (2003) 'L293D: Push-Pull Four Channel Driver with Diodes'. STMicroelectronics.
- [3] Digi-Key Electronics (n.d.) 'L293D – Bipolar Motor Driver'. Digi-Key Electronics.
- [4] Panasonic Corporation (2020) 'AA Alkaline Battery (LR6) Datasheet'. Panasonic Industry.
- [5] Adafruit Industries (2016) 'DC Hobby Gearmotor – 6V, 200 RPM Datasheet'. Adafruit.
- [6] SanDisk (2021) 'SanDisk 32GB microSDHC UHS-I Card Datasheet'. SanDisk.
- [7] Anker Innovations (2022) 'PowerCore 5000 Portable Charger – Technical Specifications'. Anker.
- [8] ELECFREAKS (n.d.) 'HC-SR04 Ultrasonic Sensor Datasheet'. ELECFREAKS.
- [9] Raspberry Pi Ltd. (n.d.) 'Camera Module 2 – Product Brief'. Raspberry Pi Ltd.

Software, Tools & Libraries

- [10] Fritzing (n.d.) 'Fritzing – Open-source Electronics Design Tool'. Retrieved 12 October 2025.
- [11] draw.io (n.d.) 'Diagramming software for creating flowcharts and technical drawings'. Retrieved 12 October 2025.
- [22] WiringPi (n.d.) 'softPwm — Software-based PWM functions for Raspberry Pi'. Retrieved from <https://projects.drogon.net/raspberry-pi/wiringpi/software-pwm-library/>
- [23] RealVNC Ltd. (n.d.) 'VNC Viewer – Remote Access Software for Raspberry Pi'. Retrieved from <https://www.realvnc.com/en/connect/download/viewer/>
- [24] Raspberry Pi Foundation (n.d.) 'Raspberry Pi OS – Official Operating System'. Retrieved from <https://www.raspberrypi.com/software/>
- [25] GNU Project (n.d.) 'Linux Terminal – Bash Command Line Shell'. Retrieved from <https://www.gnu.org/software/bash/>
- [26] Raspberry Pi Foundation (n.d.) 'Using the Terminal and SSH on Raspberry Pi'. Retrieved from <https://www.raspberrypi.com/documentation/computers/remote-access.html>

Images & Visual References

- [12] Alphasatronic (2020) 'Image of L293D Motor Driver IC'.
- [13] Raspberry Pi Foundation (n.d.) 'Image of Raspberry Pi 4 Model B'.
- [14] IndiaMART (n.d.) 'Image of Small DC Motor (3–9V, 3000 RPM)'.
- [15] Amazon (n.d.) 'Image of 2WD Smart Car Chassis Kit'.
- [16] Amazon (n.d.) 'Image of Breadboard – 400 Tie Points'.
- [17] Globalso (n.d.) 'Image of PVC Micro USB Cable'.
- [18] Duracell (n.d.) 'Image of AA Alkaline Battery Pack'.
- [19] Bing Images (n.d.) 'Image of Raspberry Pi Zero W Camera Cable'.
- [20] Bing Images (n.d.) 'Ultrasonic Sensor Measurement Diagram'.

Project & Documentation

- [21] Bilal2866 (n.d.) 'ENGG-PROJECT-3 (Week-5 Phase-1)'. GitHub. Retrieved 14 October 2025.

14. Appendices

14.1 Full Code Listing

```
#include <wiringPi.h>
#include <softPwm.h>
#include <ncurses.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

// ===== PIN MAPPING =====
//
// Motor A (Left)
// IN1 → WiringPi 0 (BCM17)
// IN2 → WiringPi 1 (BCM18)
// ENA → WiringPi 2 (BCM27)
//
// Motor B (Right)
// IN3 → WiringPi 3 (BCM22)
// IN4 → WiringPi 4 (BCM23)
// ENB → WiringPi 5 (BCM24)
//
// Ultrasonic (Mounted at BACK of car)
// TRIG → WiringPi 21 (BCM5)
// ECHO → WiringPi 26 (BCM12) — through 1k/2k divider
//
// Camera Module 2
// Connected to CSI camera port on Pi Zero W
//
// =====

// Motor pins
#define IN1 0
#define IN2 1
#define ENA 2
#define IN3 3
#define IN4 4
#define ENB 5

// Ultrasonic pins
#define TRIG 21
#define ECHO 26

// Speed presets
#define SLOW_SPEED 70
#define FAST_SPEED 90

// Motion states
#define STOPPED 0
#define FORWARD_M 1
#define BACKWARD_M 2
```

RoboRide Report

```
#define LEFT_M    3
#define RIGHT_M   4

// Auto-stop threshold (cm) for reverse
#define REVERSE_STOP_DISTANCE_CM 20.0f

int motion_state = STOPPED;

// ===== MOTOR CONTROL =====

void motor_stop(void) {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, 0);
    softPwmWrite(ENB, 0);
    delay(100); // small safety delay
}

void forward(int speed) {
    motor_stop();
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}

void backward(int speed) {
    motor_stop();
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}

void spin_left(int speed) {
    motor_stop();
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    softPwmWrite(ENA, speed);
    softPwmWrite(ENB, speed);
}

void spin_right(int speed) {
    motor_stop();
    digitalWrite(IN1, HIGH);
```

RoboRide Report

```
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH);
softPwmWrite(ENA, speed);
softPwmWrite(ENB, speed);
}

// ===== ULTRASONIC FUNCTION =====

float getDistance(void) {
    // Ensure TRIG low
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);

    // 10 µs trigger pulse
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    // Wait for echo start
    unsigned long start = micros();
    while (digitalRead(ECHO) == LOW) {
        if (micros() - start > 250000) { // 250 ms timeout
            return -1.0f;
        }
    }

    unsigned long echo_start = micros();

    // Wait for echo end
    while (digitalRead(ECHO) == HIGH) {
        if (micros() - echo_start > 250000) { // 250 ms timeout
            return -1.0f;
        }
    }

    unsigned long echo_end = micros();
    float distance = (echo_end - echo_start) / 58.0f; // HC-SR04 formula (cm)

    if (distance < 2.0f || distance > 400.0f) {
        return -1.0f;
    }

    return distance;
}

// ===== CAMERA HELPERS =====

// Ensure ~/Pictures and ~/Videos exist
void ensure_media_dirs(void) {
    const char *home = getenv("HOME");
    if (!home) home = "/home/pi";
```

RoboRide Report

```
char cmd[512];
snprintf(cmd, sizeof(cmd),
    "mkdir -p '%s/Pictures' '%s/Videos'",
    home, home);
system(cmd);
}

// Build timestamped path: dir/prefix_YYYY-MM-DD_HH-MM-SS.ext
void make_timestamp_path(char *out, size_t size,
    const char *subdir,
    const char *prefix,
    const char *ext) {
    const char *home = getenv("HOME");
    if (!home) home = "/home/pi";

    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char timestr[64];
    strftime(timestr, sizeof(timestr), "%Y-%m-%d_%H-%M-%S", t);

    snprintf(out, size, "%s/%s/%s_%s.%s",
        home, subdir, prefix, timestr, ext);
}

// Stop robot, then take a photo using libcamera-still (no preview)
void capture_photo(void) {
    motor_stop();
    motion_state = STOPPED;

    ensure_media_dirs();

    char path[512];
    make_timestamp_path(path, sizeof(path), "Pictures", "photo", "jpg");

    char cmd[768];
    // -n = no preview (works over SSH), redirect output to /dev/null
    snprintf(cmd, sizeof(cmd),
        "libcamera-still -n -o '%s' >/dev/null 2>&1",
        path);

    mvprintw(18, 0, "Camera: Capturing photo...");
    refresh();
    system(cmd);
    mvprintw(18, 0, "Camera: Photo saved to %s", path);
}

// Stop robot, then start video recording (no preview, indefinite until stopped)
void start_video(void) {
    motor_stop();
    motion_state = STOPPED;

    ensure_media_dirs();
```

RoboRide Report

```
char path[512];
make_timestamp_path(path, sizeof(path), "Videos", "video", "h264");

char cmd[768];
// -n = no preview, -t 0 = record until stopped, & = background
snprintf(cmd, sizeof(cmd),
    "libcamera-vid -n -t 0 -o '%s' >/dev/null 2>&1 &",
    path);

mvprintw(19, 0, "Camera: Starting video recording...");
refresh();
system(cmd);
mvprintw(19, 0, "Camera: Recording video to %s", path);
}

// Stop video recording
void stop_video(void) {
    motor_stop();
    motion_state = STOPPED;

    mvprintw(19, 0, "Camera: Stopping video recording...");
    refresh();
    // Kill any running libcamera-vid process
    system("pkill libcamera-vid >/dev/null 2>&1");
    mvprintw(19, 0, "Camera: Video recording stopped");
}

// ===== MAIN =====

int main(void) {
    if (wiringPiSetup() == -1) {
        printf("ERROR: wiringPi setup failed\n");
        return 1;
    }

    // Setup motor pins
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    softPwmCreate(ENA, 0, 100);
    softPwmCreate(ENB, 0, 100);
    motor_stop();

    // Setup ultrasonic pins
    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
    digitalWrite(TRIG, LOW);
    delay(50);

    // Ncurses setup
    initscr();
    cbreak();
```

RoboRide Report

```
noecho();
nodelay(stdscr, TRUE); // getch() non-blocking
keypad(stdscr, TRUE);

// UI: Title and static text
mvprintw(0, 0, "=====");
mvprintw(1, 0, "          RoboRide          ");
mvprintw(2, 0, "          Drive and Capture          ");
mvprintw(3, 0, "=====");

mvprintw(5, 0, "Controls:");
mvprintw(6, 0, " W = Forward   S = Backward");
mvprintw(7, 0, " A = Left     D = Right");
mvprintw(8, 0, " 1 = Slow Speed  2 = Fast Speed");
mvprintw(9, 0, " SPACE = Stop   Q = Quit");
mvprintw(10, 0, "Camera:");
mvprintw(11, 0, " C = Capture Photo");
mvprintw(12, 0, " V = Start Video Recording");
mvprintw(13, 0, " B = Stop Video Recording");

mvprintw(15, 0, "Status:");
mvprintw(16, 0, " Motion: STOPPED");
mvprintw(17, 0, " Speed Mode: SLOW");
mvprintw(18, 0, "Camera: Ready");
mvprintw(19, 0, "Camera: (no video recording in progress)");
mvprintw(21, 0, "Sensors:");
mvprintw(22, 0, " Distance: ---");
mvprintw(23, 0, " Reverse Obstacle: ---");

refresh();

int speed = SLOW_SPEED;
int ch;
unsigned long last_dist_ms = 0;
float last_distance = -1.0f;

while (1) {
    // ----- Handle keyboard input -----
    ch = getch();
    if (ch != ERR) {
        switch (ch) {
            case 'w': case 'W':
                forward(speed);
                motion_state = FORWARD_M;
                mvprintw(16, 0, " Motion: FORWARD          ");
                break;

            case 's': case 'S':
                backward(speed);
                motion_state = BACKWARD_M;
                mvprintw(16, 0, " Motion: BACKWARD          ");
                break;
```

RoboRide Report

```
case 'a': case 'A':
    spin_left(speed);
    motion_state = LEFT_M;
    mvprintw(16, 0, " Motion: TURN LEFT          ");
    break;

case 'd': case 'D':
    spin_right(speed);
    motion_state = RIGHT_M;
    mvprintw(16, 0, " Motion: TURN RIGHT          ");
    break;

case ' ':
    motor_stop();
    motion_state = STOPPED;
    mvprintw(16, 0, " Motion: STOPPED          ");
    break;

case '1':
    speed = SLOW_SPEED;
    mvprintw(17, 0, " Speed Mode: SLOW          ");
    break;

case '2':
    speed = FAST_SPEED;
    mvprintw(17, 0, " Speed Mode: FAST          ");
    break;

// Camera controls
case 'c': case 'C':
    capture_photo();
    break;

case 'v': case 'V':
    start_video();
    break;

case 'b': case 'B':
    stop_video();
    break;

case 'q': case 'Q':
    motor_stop();
    stop_video(); // just in case
    endwin();
    return 0;

default:
    break;
}
}
```

// ----- Ultrasonic reading every 2 seconds -----

RoboRide Report

```
unsigned long now = millis();
if (now - last_dist_ms > 2000) {
    last_dist_ms = now;

    float d = getDistance();
    last_distance = d;

    if (d < 0) {
        mvprintw(22, 0, " Distance: --- (no echo)          ");
        mvprintw(23, 0, " Reverse Obstacle: UNKNOWN          ");
    } else {
        mvprintw(22, 0, " Distance: %6.2f cm          ", d);

        // Auto-stop only when reversing and obstacle is too close
        if (motion_state == BACKWARD_M && d < REVERSE_STOP_DISTANCE_CM) {
            motor_stop();
            motion_state = STOPPED;
            mvprintw(16, 0, " Motion: STOPPED (AUTO REVERSE STOP)  ");
            mvprintw(23, 0, " Reverse Obstacle: TOO CLOSE!          ");
        } else {
            mvprintw(23, 0, " Reverse Obstacle: CLEAR          ");
        }
    }
}

refresh();
delay(20); // avoid maxing CPU
}

endwin();
return 0;
}
```

14.2 Explanation of code for better understanding

- **Header Inclusions:**

The program begins with the inclusion of essential libraries:

- wiringPi.h → Provides GPIO control for Raspberry Pi pins.
- softPwm.h → Enables software-based PWM generation for motor speed control.
- ncurses.h → Handles terminal-based UI and keyboard input.
- stdio.h, stdlib.h, string.h, time.h → Standard C libraries for input/output, system calls, string manipulation, and timestamp generation.

- **Pin Mapping Definitions:**

Each motor and sensor pin is mapped using #define statements.

- Motor A (Left) → IN1, IN2, ENA
- Motor B (Right) → IN3, IN4, ENB
- Ultrasonic sensor → TRIG, ECHO

This improves readability and allows easy reconfiguration without changing logic everywhere in the code.

- **Speed Constants:**

Two speed presets are defined:

- SLOW_SPEED = 70
- FAST_SPEED = 90

These values represent PWM duty cycles (0–100) for controlled motor operation.

- **Motion States:**

Integer constants (STOPPED, FORWARD_M, BACKWARD_M, LEFT_M, RIGHT_M) track the robot's current motion state. This makes it easier to update UI and enforce safety checks.

- **Motor Control Functions:**

Modular functions handle all movement operations:

- motor_stop() → Stops both motors safely by setting all control pins LOW and PWM to 0.
- forward(speed) → Drives both motors forward at the given PWM duty cycle.
- backward(speed) → Reverses both motors.
- spin_left(speed) → Left spin by reversing left motor and driving right motor forward.
- spin_right(speed) → Right spin by reversing right motor and driving left motor forward.

Each function calls motor_stop() first to prevent conflicting signals.

- **Ultrasonic Sensor Function:**

- getDistance() → Sends a 10 μ s trigger pulse, waits for echo, and calculates distance using the HC-SR04 formula.
- Includes timeout handling (250 ms) to avoid infinite loops.
- Filters out invalid readings (400 cm).
- Used to auto-stop the robot when reversing too close to an obstacle.

- **Camera Helper Functions:**
 - `ensure_media_dirs()` → Creates ~/Pictures and ~/Videos directories if they don't exist.
 - `make_timestamp_path()` → Builds timestamped file paths for photos and videos.
 - `capture_photo()` → Stops the robot, then runs `libcamera-still` to capture a photo.
 - `start_video()` → Stops the robot, then runs `libcamera-vid` in background to record video.
 - `stop_video()` → Stops motors and kills any running `libcamera-vid` process. These functions integrate Raspberry Pi's camera module with the robot.
- **Ncurses UI Setup:**
 - Initializes `ncurses` with `initscr()`, `cbreak()`, `noecho()`, and `nodelay()` for non-blocking input.
 - Displays a text-based control panel with instructions, motion state, speed, distance, and camera status.
 - Updates UI dynamically as the robot moves or captures media.
- **Main Function Logic:**
 - Initializes GPIO pins and PWM channels.
 - Sets up ultrasonic sensor pins.
 - Enters a continuous loop where:
 - Distance is polled every ~100 ms.
 - Auto-stop is triggered if reversing too close to an obstacle.
 - Keyboard input (`getch()`) controls motion, speed, photo/video capture, or program exit.
 - Supported keys:
 - Arrow keys → Forward, Backward, Spin Left, Spin Right
 - Space → Stop
 - 1 / 2 → Change speed (slow/fast)
 - P → Capture photo
 - V → Start video
 - B → Stop video
 - Q → Quit program safely
- **Input Handling and Safety:**
 - Motion always stops before changing direction.
 - Reverse motion is auto-stopped if an obstacle is detected within 20 cm.
 - Camera operations also stop the robot before capturing.
 - Safety delays (`delay(100)`) prevent motor driver conflicts.
- **Code Organization and Readability:**
 - Functions are modular and self-contained.
 - Clear naming conventions (`forward`, `backward`, `spin_left`) improve readability.
 - Consistent commenting explains each section.
 - Easily extendable for future features (e.g., wireless control, additional sensors).

14.3 Additional Diagrams or Tables

- Raspberry pi Schematic:

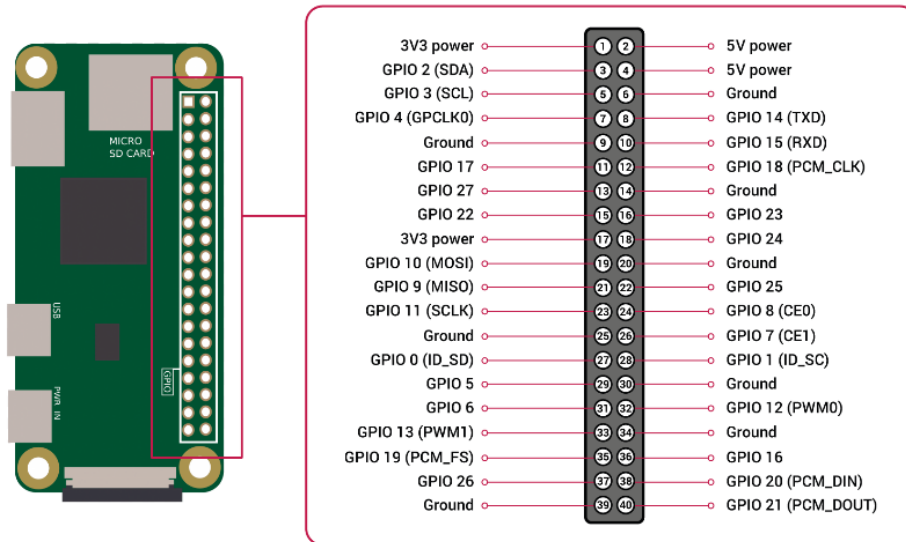


Figure 20: IC Schematic for wiring help

- Footprint for IC:

Footprint

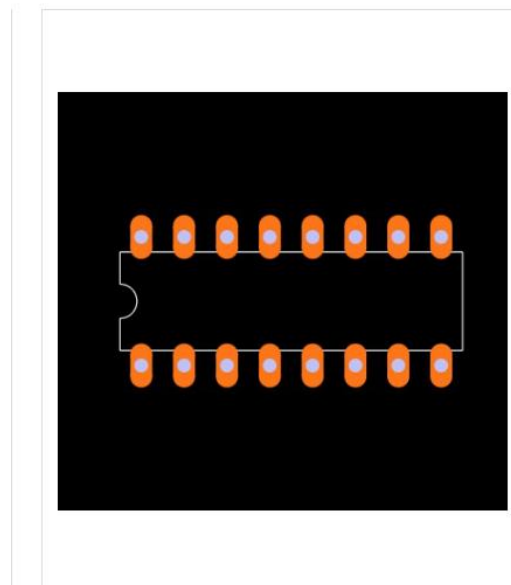


Figure 21: Footprint of L293D IC

RoboRide Report

- Extra pictures and diagram from the RoboRide building process:

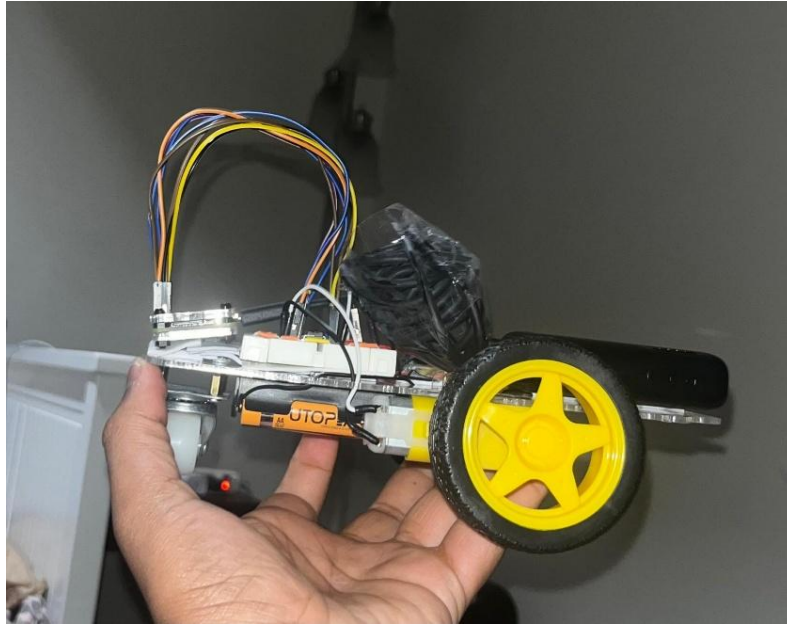


Figure 22: Another angle of RoboRide from Phase 1.

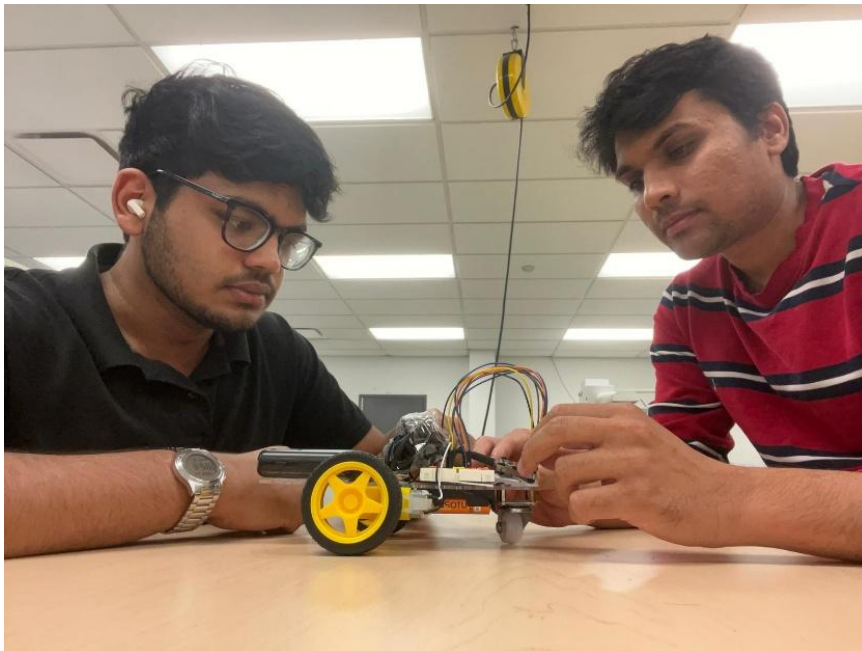


Figure 23: Group working on RoboRide.

RoboRide Report

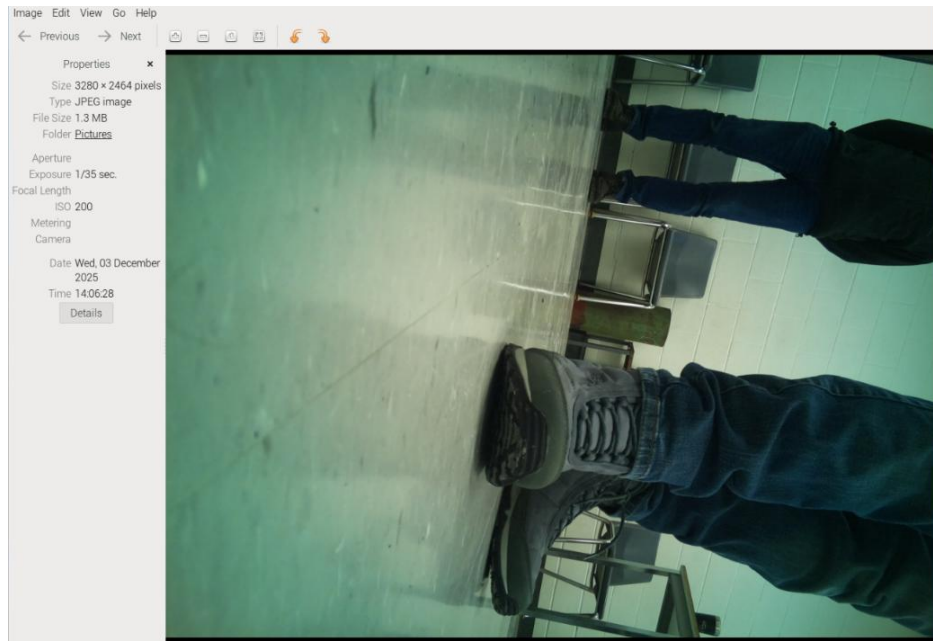


Figure 24: Picture taken from RoboRide during Final Demo



Figure 25: Ultrasonic test at high temperature.

RoboRide Report



Figure 26: Ultrasonic sensor testing at low temperature.

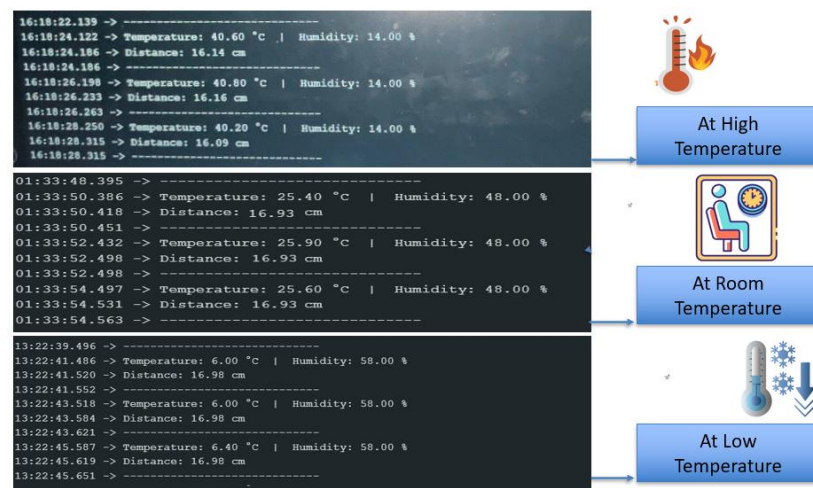


Figure 27: Readings from HC-SR04 Test.

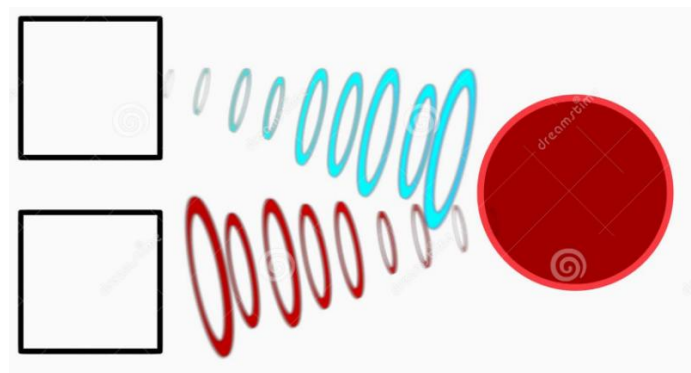


Figure 28: Diagram used in HC-SR04 presentation.



Figure 29: RoboRide Rear View.

RoboRide Week 10 Progress

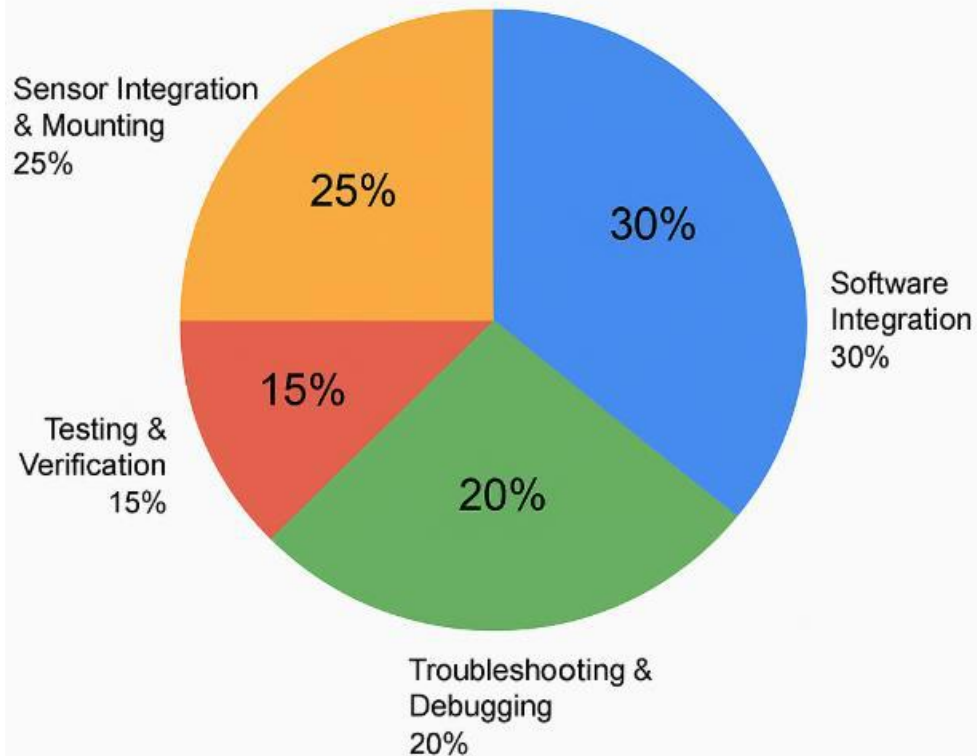


Figure 30: Pie Chart from Troubleshoot Report.

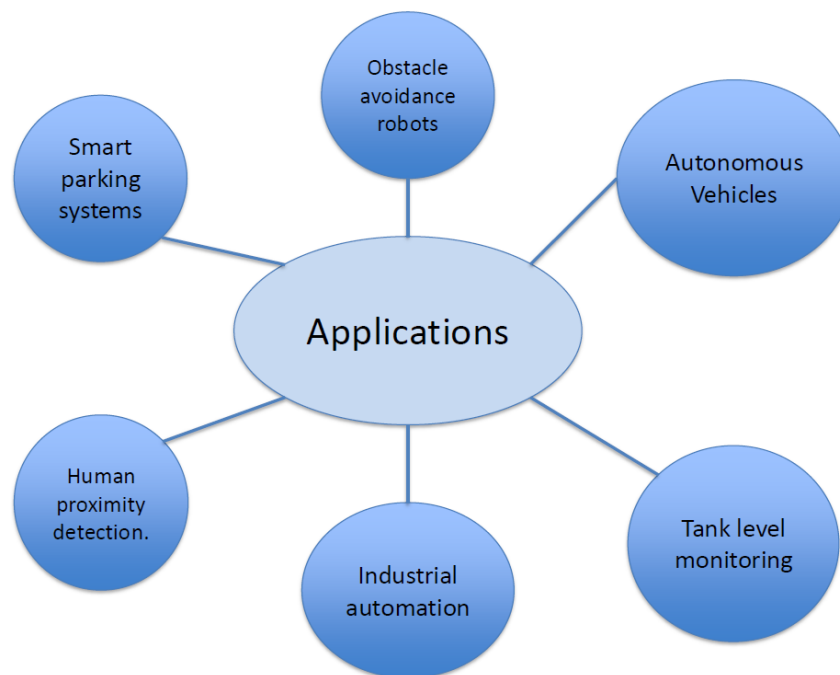


Figure 31: Diagram used in Ultrasonic Sensor Presentation.

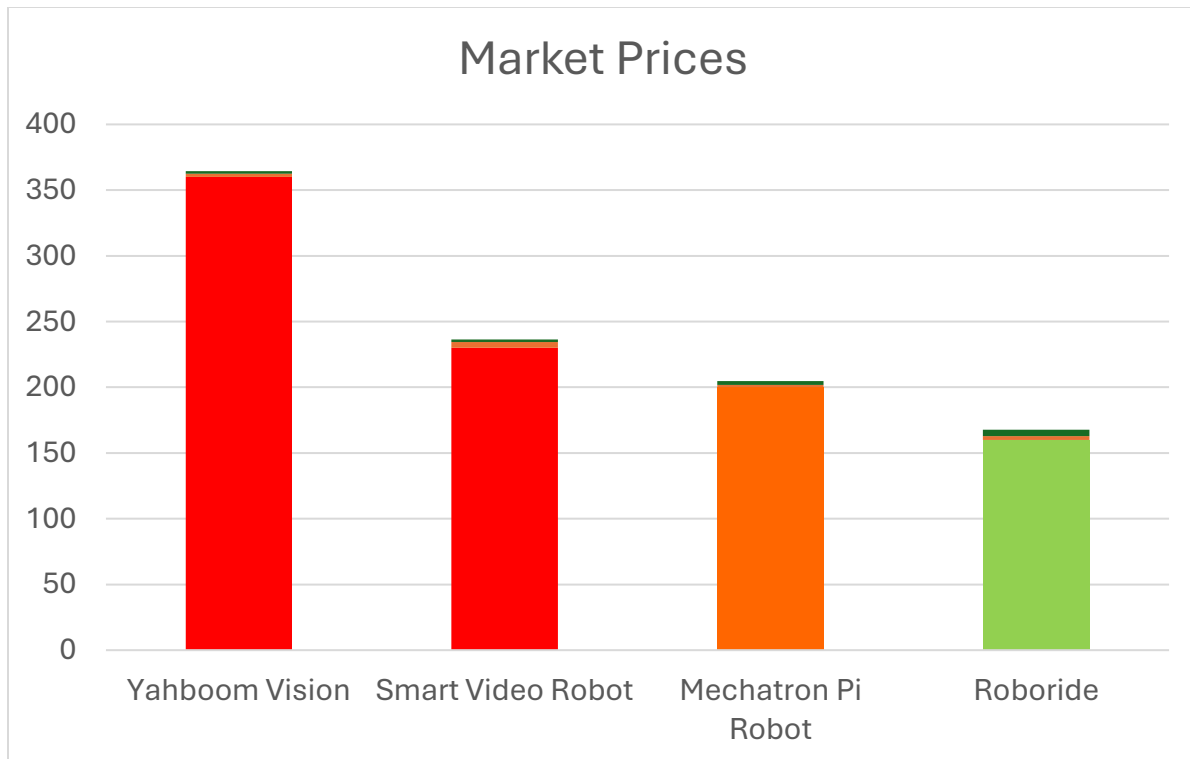


Figure 32: Diagram of Market Prices of similar robot from Project Pitch.

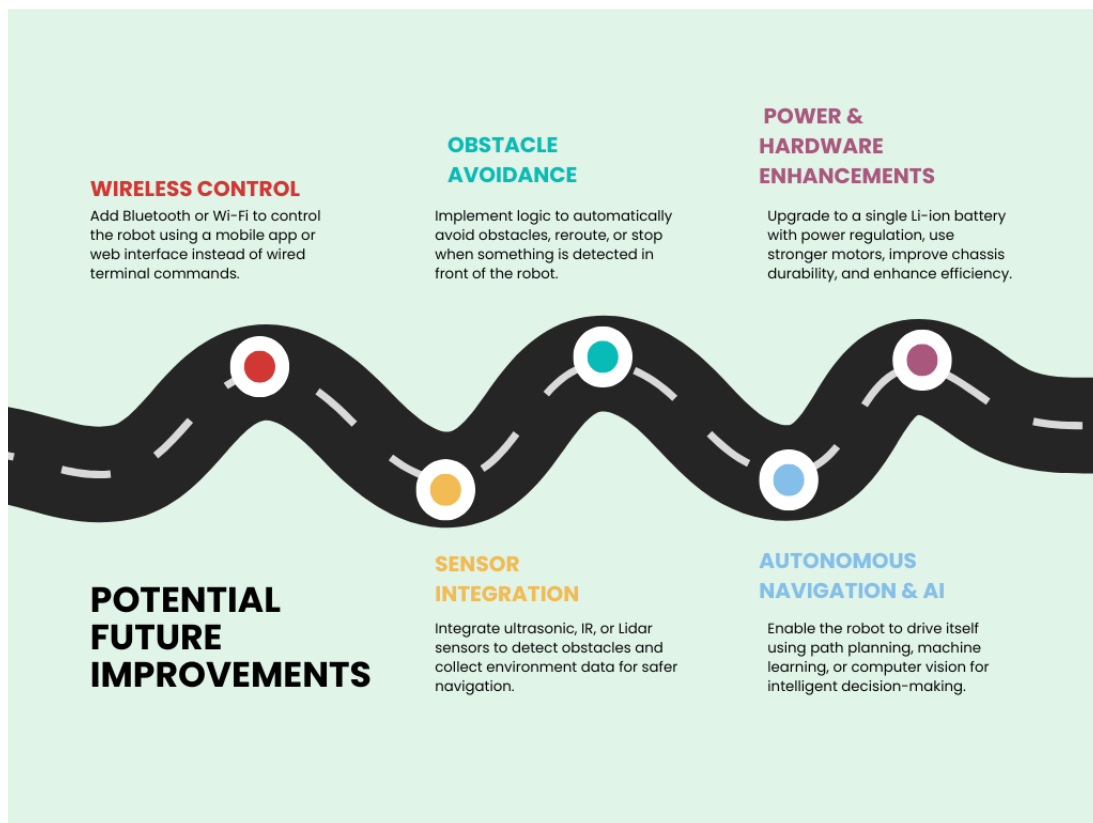


Figure 33: Future Improvements from Phase 1